Deepotus Video Gen

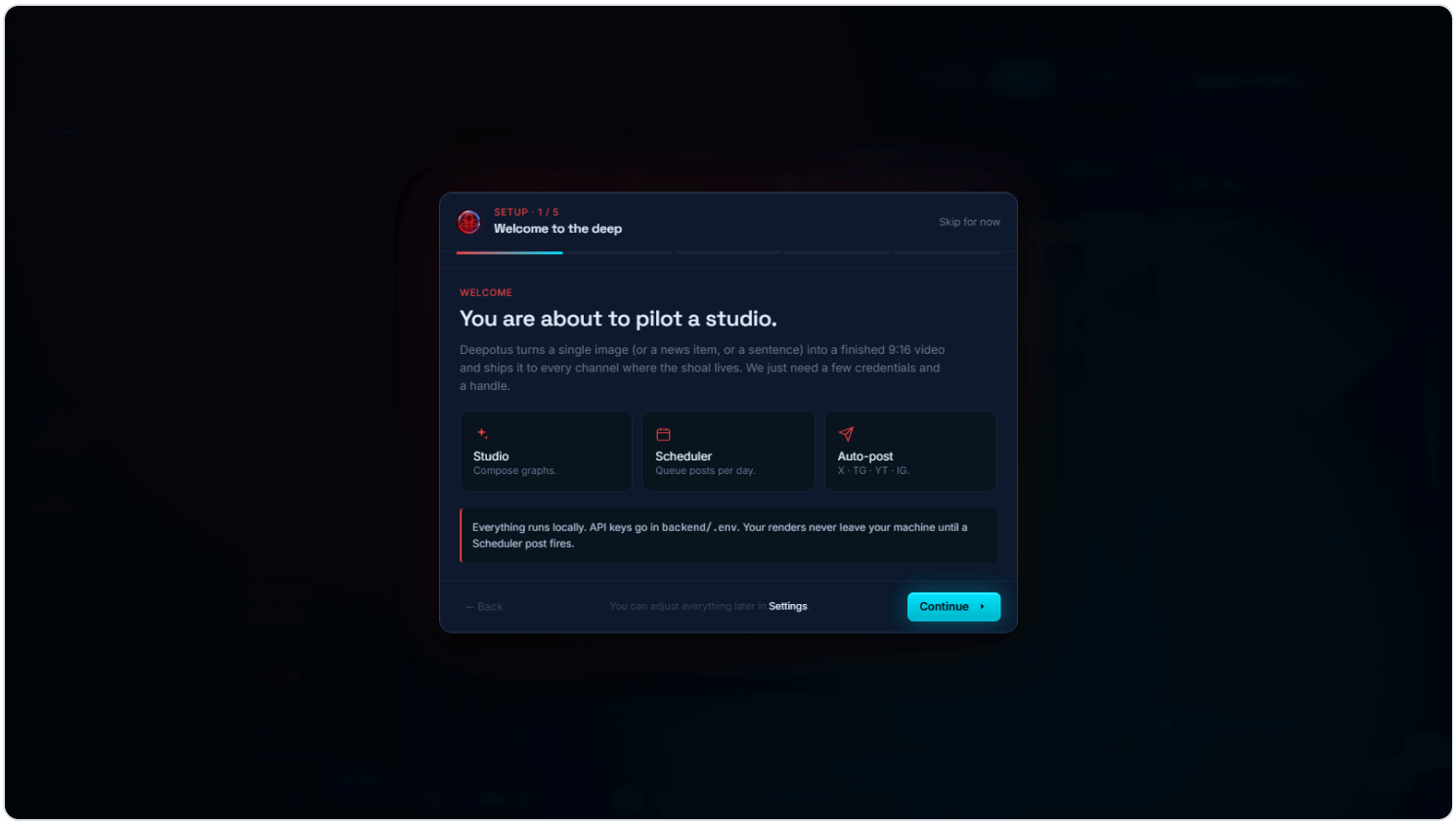
Complete guide — from double-click to your first auto-published post. **v2.6.0** · Version française

> **Never touched an editing tool?** Good — this guide is written for you. Deepotus Video Gen turns an idea into a post-ready vertical video, with no software to learn. Everything runs **on your computer**; you plug in your own API keys (you pay your providers, not the app).
>
> **Five ways to make a video**, from simplest to richest:
>
> ① a still image animated into a **cinematic clip** (Seedance) · ② an **avatar that speaks** your script (HeyGen) · ③ **your own footage** shot on a phone (UGC) · ④ a **composition** mixing several sources (Templates or Studio) · ⑤ a whole **narrated novel chapter** turned into an illustrated video (Episodes). Add **voiceover and background music** on top, then the **Scheduler** plans and publishes on its own.

---

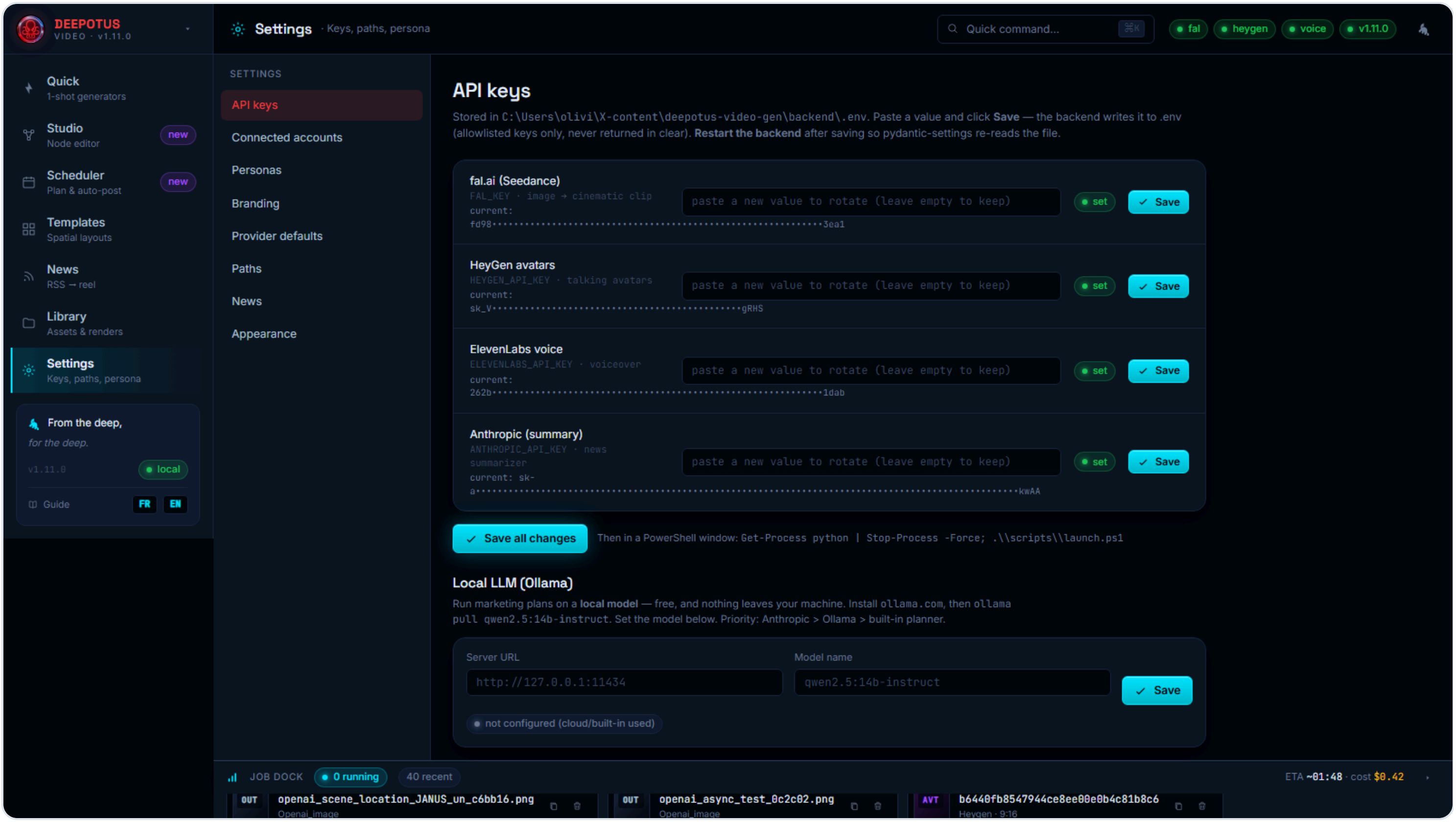
0 Start & API keys

Double-click the **Deepotus Video Gen** icon on the Desktop. The app starts silently (no black window), then your browser opens on the studio. On first launch a 5-step wizard guides you: persona, providers, channels. Replay it anytime (command palette `⌘K` → "Replay onboarding").



The first-run wizard. The "Generators" step lists your providers, including the **Ollama (local LLM)** option.

1. The app is **BYO keys**: you use your own API accounts. Only one is **required** to start — **fal.ai** (it makes both images *and* video clips). Create it at `fal.ai/dashboard/keys`.
2. Paste your keys in **Settings** → **API keys**, hit *Save*, then restart the app. The pills top-right (`fal` / `heygen` / `voice`) turn **green** when a key is active.



Settings → API keys: each key has a field, a status and a Save button. At the bottom, the **Local LLM (Ollama)** card (chapter 11).

How much does it cost? (ballpark)

Action	Service	Indicative cost
Create an image	fal.ai (FLUX)	≈ $0.003
10 s cinematic clip	fal.ai (Seedance)	≈ $0.30–0.60
1 min talking avatar	HeyGen	≈ 6 credits
Voiceover / narration	ElevenLabs	≈ per characters · free tier available
Upload your own video / sound	—	free
News summary / marketing plan	Anthropic / OpenAI / Gemini / Ollama	≈ $0.01 / free (Ollama)
Publish to Telegram / X	—	free / your X credits

Which key unlocks what?

API key	Required?	What it unlocks
fal.ai	**Required**	Images (FLUX) *and* video clips (Seedance) — the only key you need to start.
HeyGen	Optional	Talking avatars (video type ②) and avatar + clip compositions.
ElevenLabs	Optional	Voiceover / narration — for your UGC videos **and the Episodes feature** (chapter 7), saved straight to the audio Library.
Anthropic, OpenAI or Gemini (any one is enough)	Optional	News summaries, marketing plans **and genuinely AI-written scripts** (News, Studio & Episode storyboards). Without a key: deterministic brand-template version.
Ollama (local LLM)	Optional	Same AI uses as above, **100% local and free** (chapter 11).
X (4 keys)	Optional	Auto-publishing to X/Twitter (chapter 12).
Telegram (bot token)	Optional	Auto-publishing to a Telegram channel.

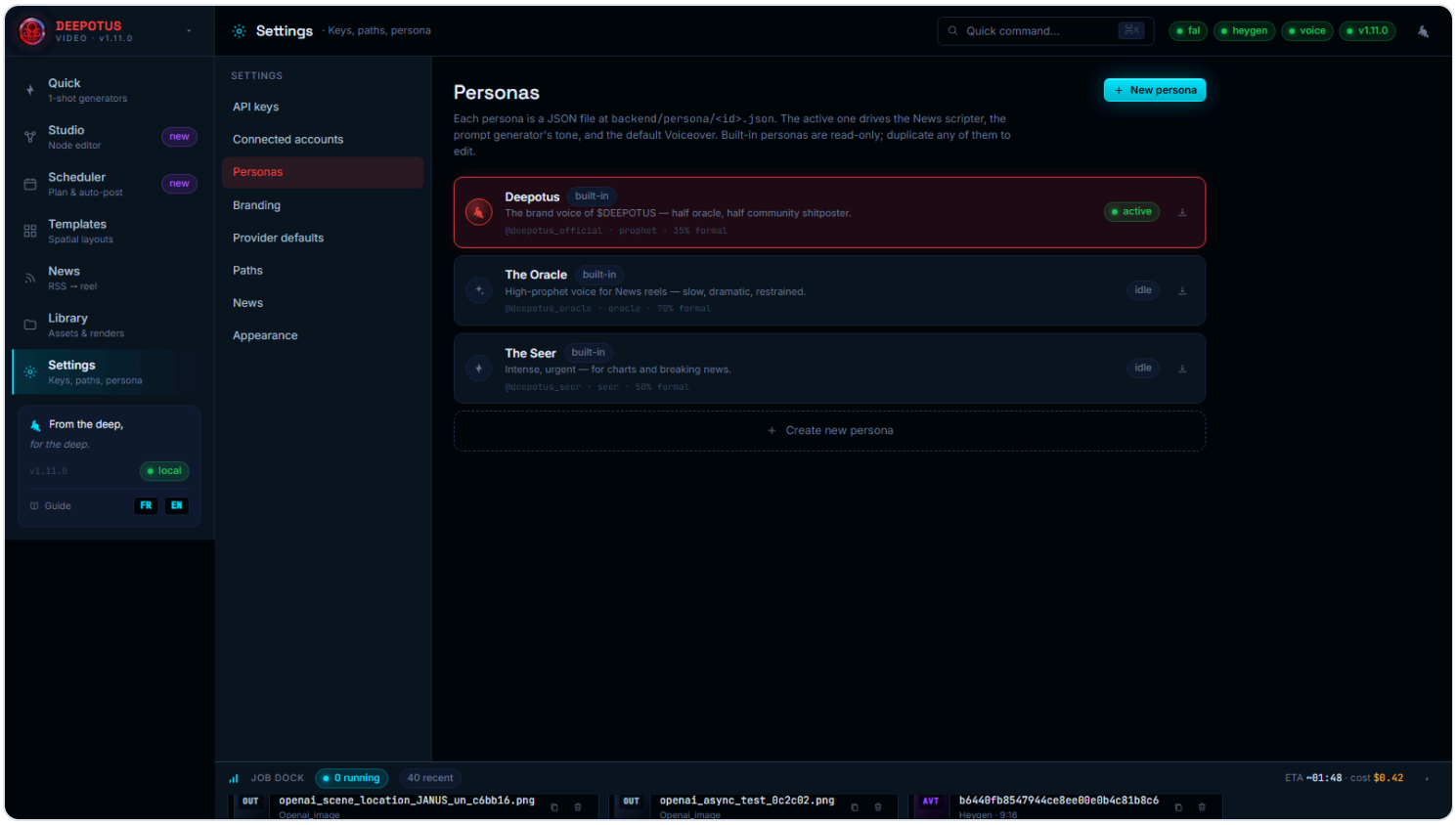
Tip: start with **fal.ai** alone, then add keys as you explore avatars, voiceover, AI and publishing.

> Costs are billed by your providers, not the app. Watch your credits on their dashboards. Your keys, files and database stay **on your machine** (`%LOCALAPPDATA%\DeepotusVideoGenData`) — they survive updates and reinstalls.

---

1 Set up a persona

The persona is your **brand voice**: it feeds marketing plans, avatar scripts and the prompt generator. Go to **Settings** → **Personas**.



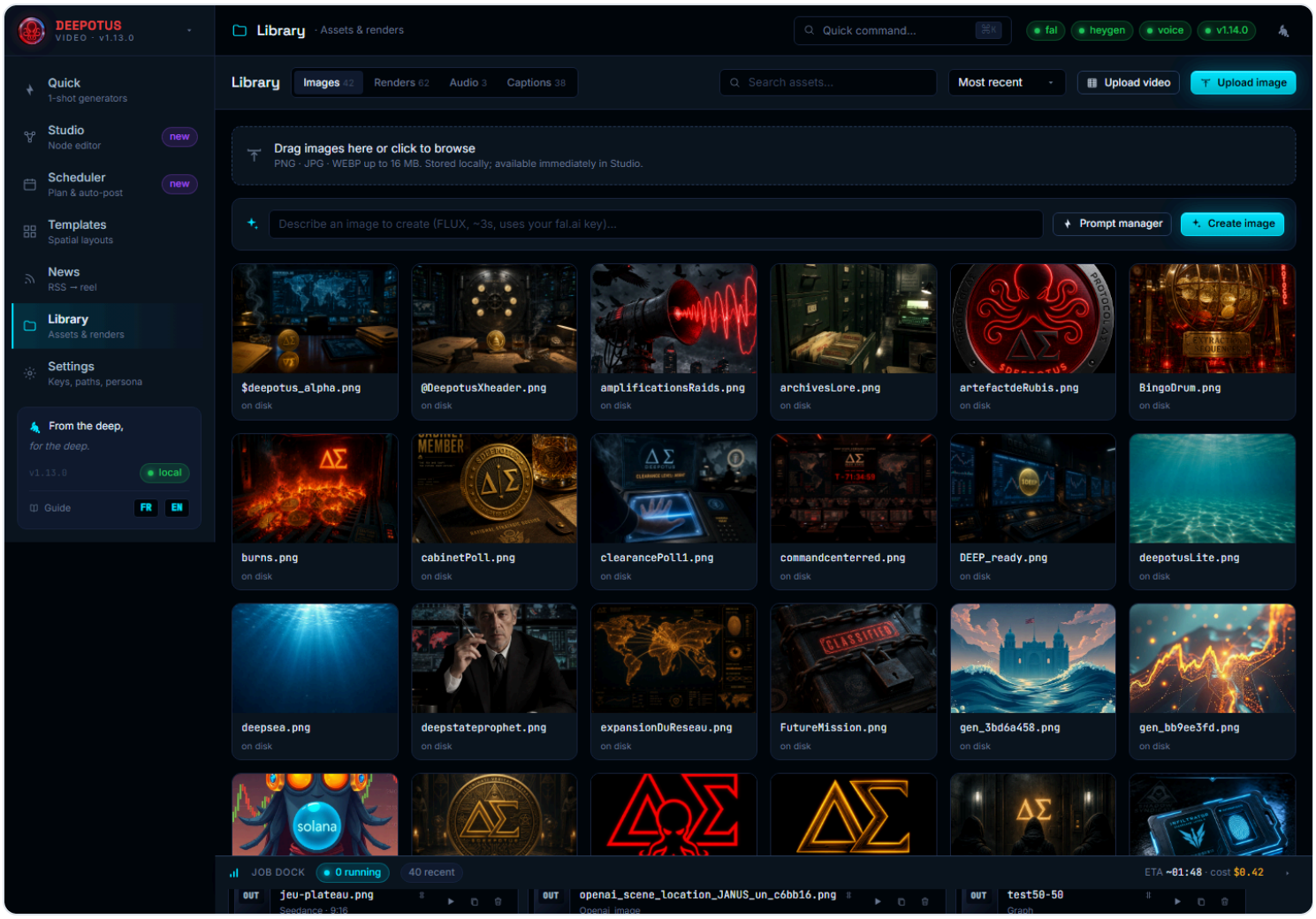
Settings → Personas: create, edit and activate your brand voice.

1. Click *New persona*: name, tone (e.g. "prophetic, playful"), audience, voice mode.
2. Click the card to **activate** it — the active persona is echoed across the generation screens.

---

2 Create your first images

Library tab: your library of images, video renders, sounds and captions. Everything lands here.



Library: create an image (FLUX), the **Prompt manager**, **Upload image** and **Upload video** (your own clip). Hover a render to play it.

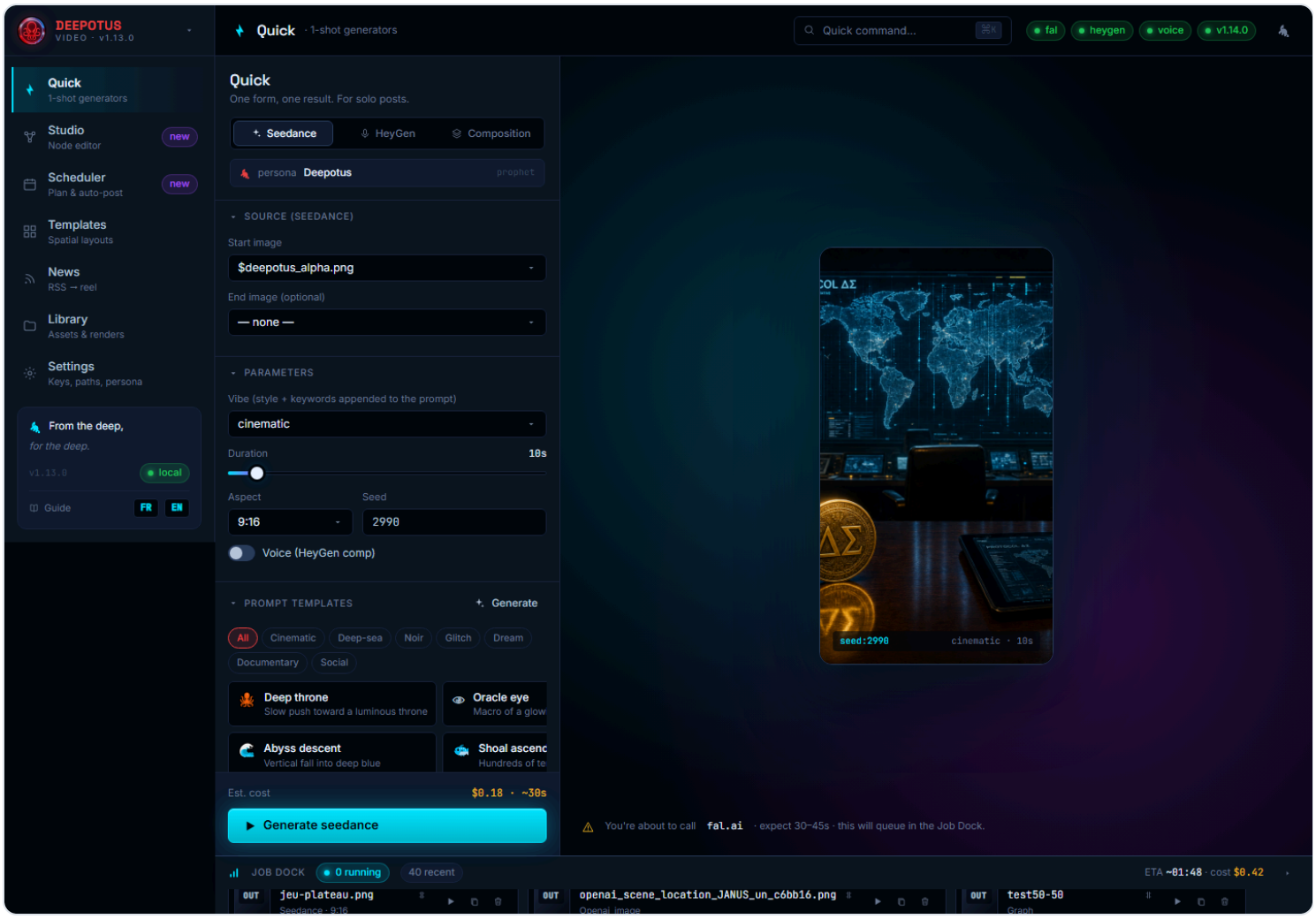
1. "*Describe an image to create*" field → describe in one line, Enter. The FLUX image (~3 s) lands straight in the library.
2. Out of inspiration? Click **Prompt manager**: pick ingredients (subject, mood, light, motion…), "Random", or "Refine with AI", then *Use this prompt* — it **overwrites** the field for a more personal result.
3. Already have visuals? **Upload image** (or drag-and-drop anywhere). For your videos, see chapter 5; for sounds, chapter 6.

> Prompt examples: "neon-red octopus mascot, abyssal background, cinematic" · "stylized market chart, bioluminescent mood" · "underwater throne, light rays, 9:16".

---

3 Video type ① — the cinematic clip (Seedance)

Quick tab, **Seedance** mode: one image + a prompt = a vertical cinematic clip.



Quick / Seedance: sources on the left, real-format preview on the right, estimated cost before you launch.

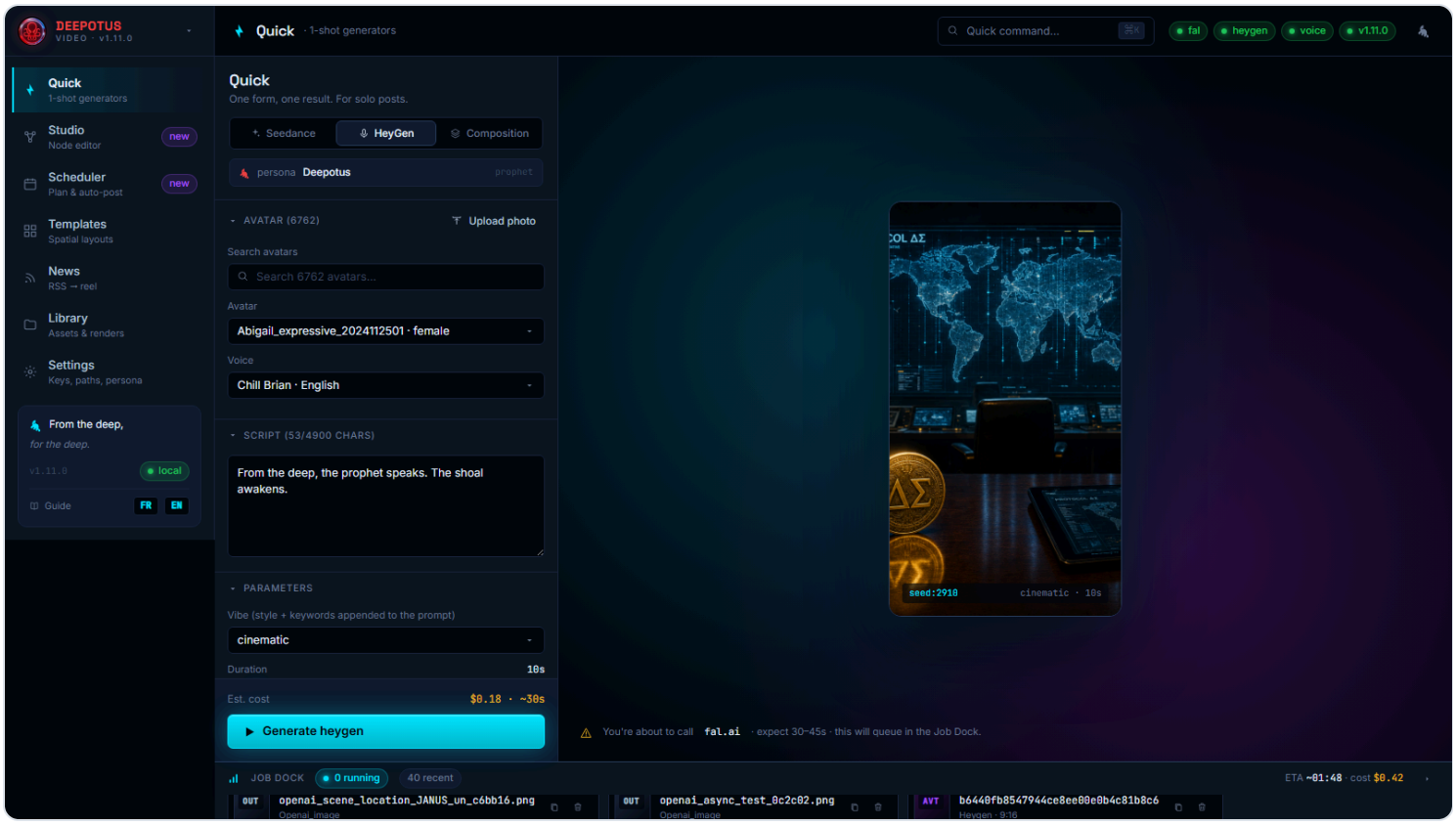
1. Pick the start image. Optional: an end image for a transition.
2. Write the motion prompt, grab a *template*, or open the **prompt generator** (it overwrites the default prompt). Set style, duration (5–60 s), format (9:16 default).
3. *Generate* → follow progress in the **Job Dock** (bottom bar). 30–45 s later the render is in Library.

> **Rename a render:** in the Job Dock, click the pencil 🖍 next to the name (or play it first with ▶). The new name instantly reflects in the Library and pickers.

---

4 Video type ② — the talking avatar (HeyGen)

Still in **Quick**, **HeyGen** mode: a script read by an avatar.



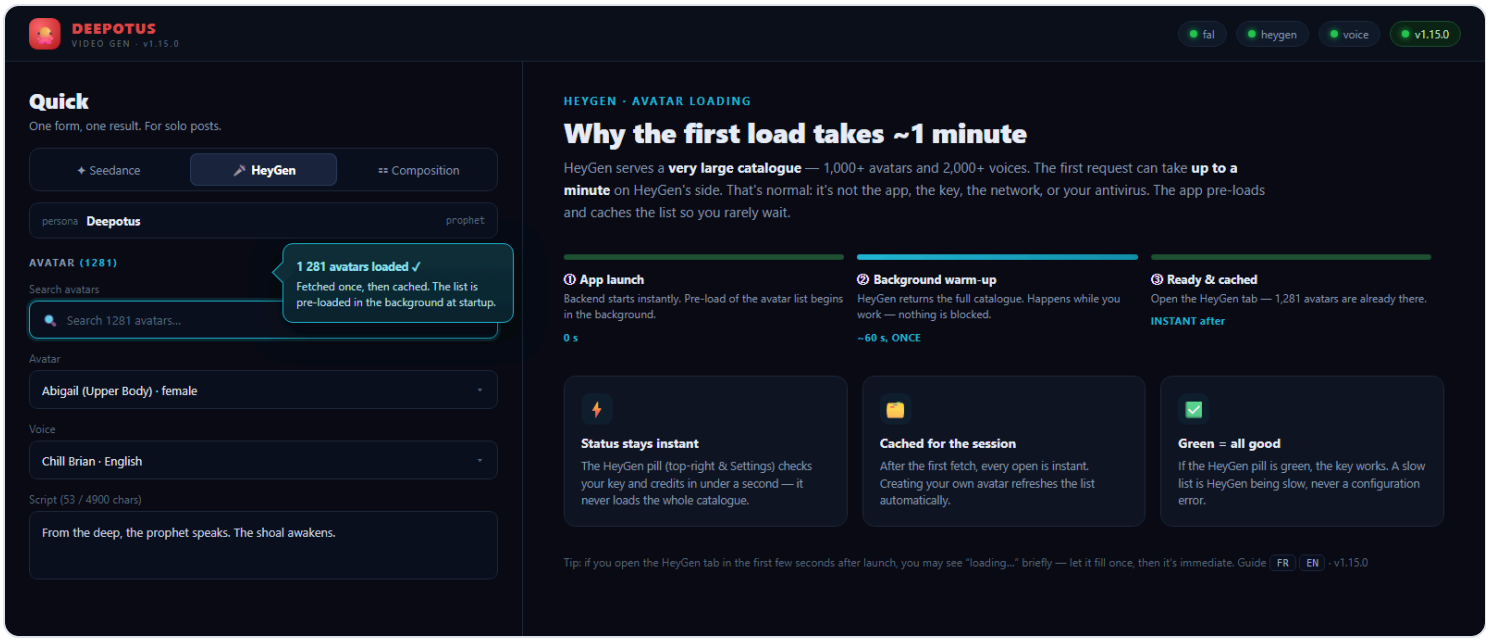
Quick / HeyGen: avatar + voice selection, upload your photo for a custom avatar, script.

1. Pick an avatar and a voice from the lists (search field available).
2. **Create your own avatar:** *Upload photo* → your photo (PNG/JPG) becomes a talking avatar in ~30 s, auto-selected.
3. Write the script (up to 4,900 chars), *Generate*. Allow 1 to 3 minutes.

⏱ The first avatar load — why it takes ~1 minute v1.15

HeyGen gives you a **very large catalogue** — often **1,000+ avatars** and 2,000 voices. The very first time the app requests this list, HeyGen's server can take **up to a minute** to respond. **This is HeyGen's own service being slow** — not an app bug, not a key problem, and not a network or antivirus issue.

So you don't have to wait, the app **pre-loads the list in the background at startup**: by the time you reach the HeyGen tab, the avatars are usually **already there**. Once fetched, the list is **cached** — every later open is **instant**.



Left: the HeyGen tab once loaded (1,281 avatars, search field, avatar & voice selectors). Right: why the first load takes ~1 min — pre-loaded at startup, then instant.

> **If you open the tab right after launch:** the list may briefly show "loading..." (up to ~1 min, **once**). Let it fill, then everything is instant — including after you create your own avatar (the list refreshes itself). The **HeyGen status** pill (top-right and in Settings) stays **instant** at all times: it checks your key and credits without loading the whole catalogue.

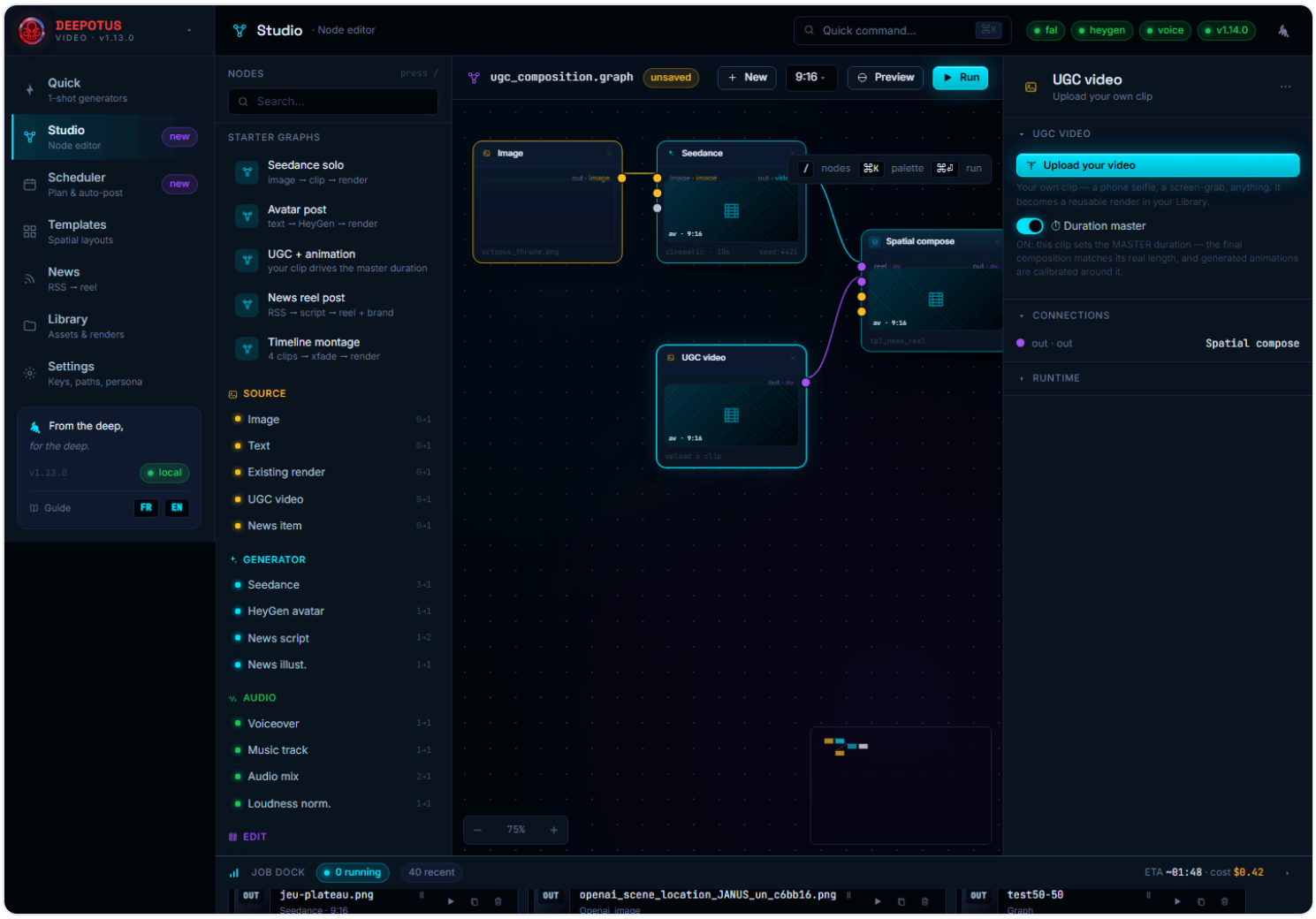
Composition mode combines both: a Seedance clip + a HeyGen avatar in one video (sequential or split-screen). **Anti-cut** is automatic: the avatar always finishes its sentence.

---

5 Video type ③ — your own video (UGC)

Many creators film themselves on a phone — a selfie, a reaction, an unboxing. Deepotus welcomes these raw clips (**UGC**, user-generated content) and marries them to your animations.

1. **Simplest:** Library → **Upload video**. Your clip is imported, its real duration is measured, and it appears in the *Renders* tab like any render — reusable everywhere (Scheduler, Studio).
2. **In the Studio:** drag a **UGC video** node onto the canvas, click it, then *Upload your video*. A thumbnail fills the node.



Studio: the **"UGC + animation"** starter. On the right, the **UGC video** node with the ⏱ **Duration master** switch.

The "master" duration — the key idea

When you turn on ⏱ **Duration master** on your clip, **its real length** drives the final video. Generated animations (Seedance) are then calibrated around your clip — no more guessing "how many seconds?". You talk for 12 seconds to camera → the composition is 12 seconds, your illustration animation follows. It's the equivalent, for your own footage, of the avatar anti-cut.

> **Quick recipe:** Studio → "UGC + animation" starter → upload your clip in the UGC video node (master on) → connect an Image + Seedance as the animated half → *Run*. Output: your video at the bottom, the animation on top, all at your clip's length.

---

6 Audio, voiceover & music v1.15.6

Sound makes a video. Deepotus now has a full **audio side**: import your own sounds, turn a script into a **voiceover**, and lay a **background music** track under any render.

The audio Library

The **Library** has an **Audio** tab next to Images and Renders. It gathers every sound: imported files, generated voiceovers and music tracks.

1. **Import a sound:** the *Upload sound* button (or drag-and-drop onto the Audio tab) → pick an `.mp3` / `.wav` / `.m4a` (and more). Its duration is measured; it appears in the Audio tab, playable inline, reusable everywhere.
2. The Audio tab also lists every **voiceover you generate**, so your narrations and your imported tracks live side by side.

> The Audio tab links a few **royalty-free music banks** (Pixabay Music, Freesound, Free Music Archive, YouTube Audio Library). Only add music you have the right to use — Deepotus cannot ship licensed music for you.

Voiceover (ElevenLabs) v1.17.0

Turn text into a natural narration with your **ElevenLabs** key.

1. In **Quick**, open the **Voice Over** tab: a voiceover **on its own** (no image, no video), perfect to narrate over your own visuals. Write a short script (up to 2,500 characters — for longer narrations use Chapters), pick the **language**; the counter and the **estimated cost** update live.
2. Pick a voice from the **scrollable list** (▶ to preview, filter field) or keep the **app default voice**, and save your voice + language combos as reusable **castings** (★ Save). *Generate voice* → the mp3 plays inline and is **saved to Library → Audio**, ready to reuse (Studio, Episodes, Scheduler).

> **ElevenLabs free plan:** the built-in "premade" voices (George, Sarah…) work via the API. Custom "library / community" voices require a paid ElevenLabs plan — the app tells you clearly when a chosen voice is paid-only.

Background music (BGM)

Drop a music bed under your video — the app mixes it for you.

1. **In the Studio:** add a **Music track** node, pick a track from the audio Library (with an inline preview), set its **volume** and toggle **loop to duration**. Wire it into the Render node.
2. **On the simple render too:** you can attach a background track before generating, no Studio needed.
3. The track is automatically **looped or trimmed to the exact length** of the final video, then mixed under the voice/clip. *Run* — your render comes back with music.

7 Video type ⑤ — Episodes (a narrated novel → video) v1.15.6

You have a story to tell — a chapter, a lore drop, a serialised novel. The **Episodes** page turns a block of text into a **narrated, illustrated vertical video**, scene by scene, ready for the Scheduler. Open it from the sidebar (**Episodes**). A clickable stepper walks you through four steps.

Step 1 — the chapter & its narration

1. Give the episode a **title**, then **paste your text** or *Upload* a `.txt`, `.docx` or `.pdf` — the text is extracted for you.
2. Pick a **voice** and a **language** (preview a voice with ▶).
3. *Generate narration* → the whole chapter is read by ElevenLabs and saved to the audio Library. Long chapters are split and stitched back together automatically.

Step 2 — the storyboard (scenes)

A video needs shots. The app cuts your chapter into **scenes**, each one a bit of text + its own illustration.

1. Choose the split: **By paragraph** (verbatim, instant) or **By AI** (the AI groups the text into visual beats and writes a rich illustration prompt for each). You can **generate both and compare** before locking one in.
2. **Edit freely:** tweak each scene's text and its illustration prompt, reorder them (↑ ↓), add or remove scenes (+ / ✕).

Step 3 — illustrations & motion

1. *Generate all illustrations* (or one at a time): one image per scene, with your chosen image model (FLUX or GPT Image).
2. Per scene, pick the **motion**: **Ken Burns** (a slow cinematic zoom/pan, default), **Seedance** (a true animated clip) or **Still** (a fixed image).

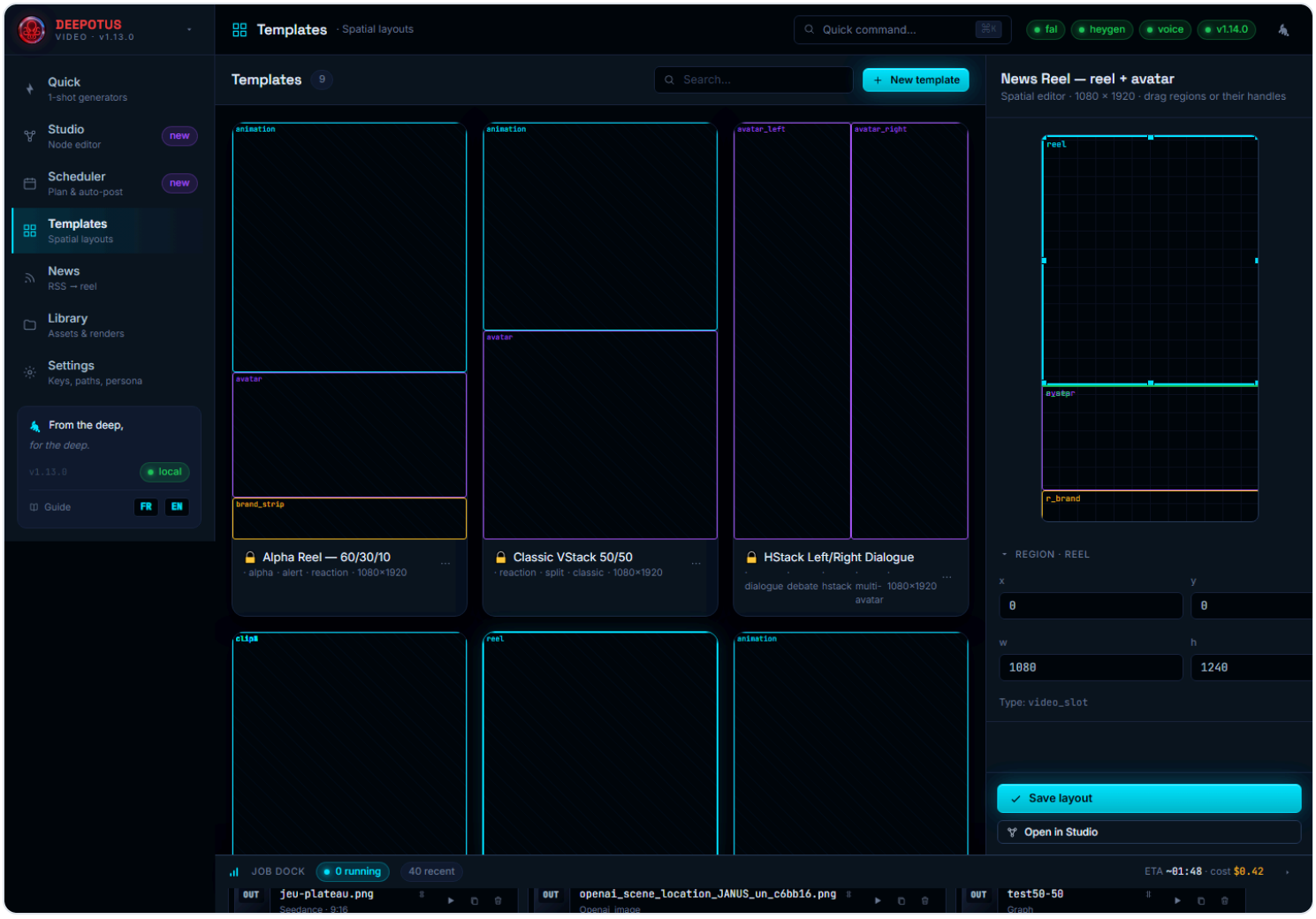
Step 4 — assemble & send

1. *Assemble the episode*: each scene's illustration is animated over its slice of narration, then all scenes are concatenated into one vertical video — follow it in the Job Dock.
2. Preview the result, then *Send to Scheduler* → a draft post on **YouTube** and **Instagram**, ready to caption and schedule.

> Serialising a novel? Keep the same persona and voice across the series for consistency, then publish one episode per day from the Scheduler.

8 Video type ④ — compose with Templates

Templates tab: ready-made 9:16 layouts (news reel + avatar + brand strip, split-screen, PIP corner, triptych…). Each thumbnail reflects its **real region layout** — they're all different.

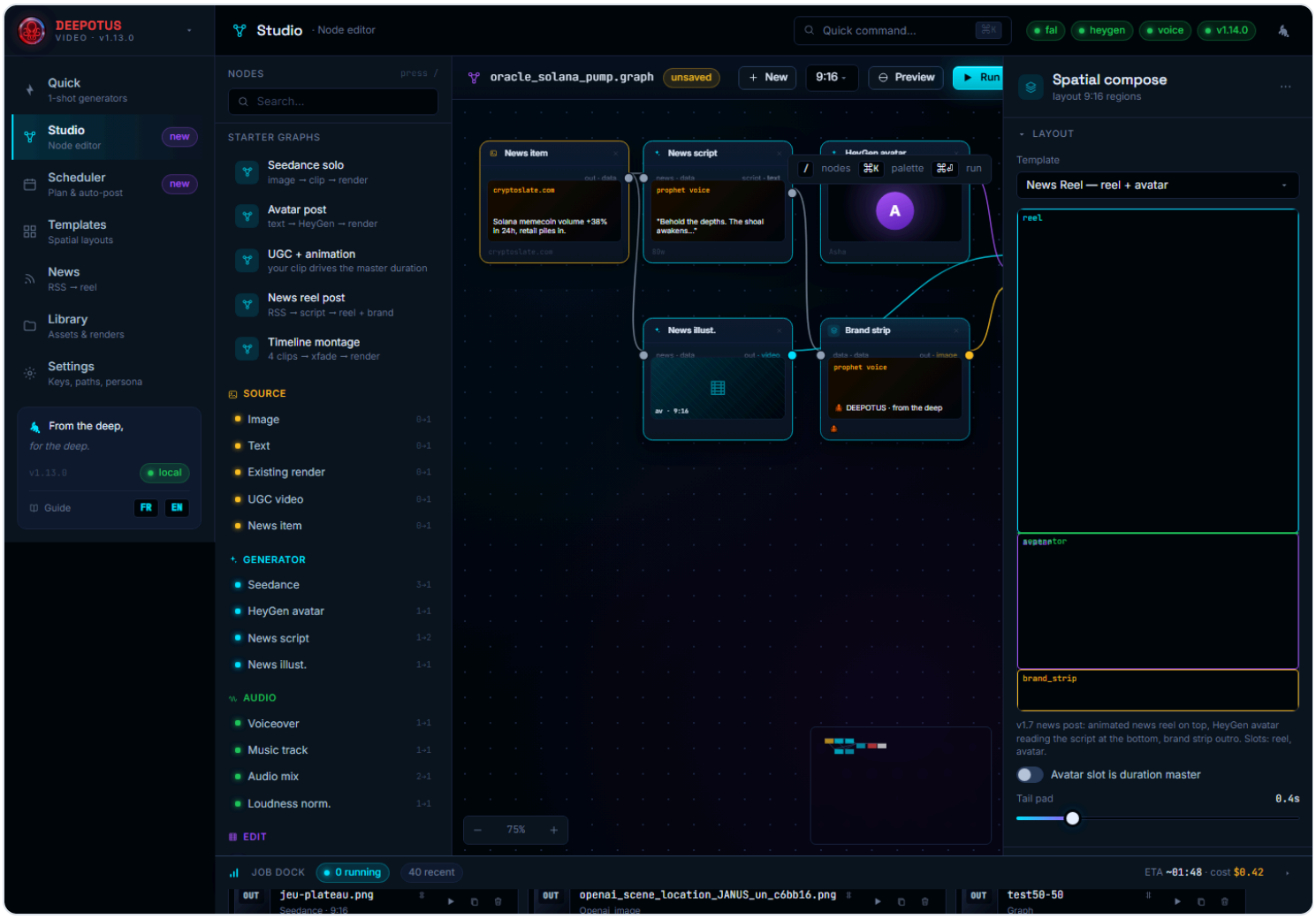


Templates: gallery of layouts (distinct thumbnails), spatial editor on the right (drag regions or their handles).

1. Click a thumbnail: the right editor shows its regions. Move them, resize them.
2. *Save layout* stores your version — it **appears immediately** in the gallery. *New template* creates a blank one.
3. *Open in Studio* switches to the node editor for deeper editing.

9 Compose with the Studio (node editor) EXPANDED

The **Studio** is the advanced workshop: you wire *nodes* (sources → generators → montage → render) like a visual recipe. Don't worry — always start from a ready-made starter graph.



Studio: the node palette on the left, the canvas in the middle, the inspector on the right. Top bar: **New**, **Preview**, **Run**.

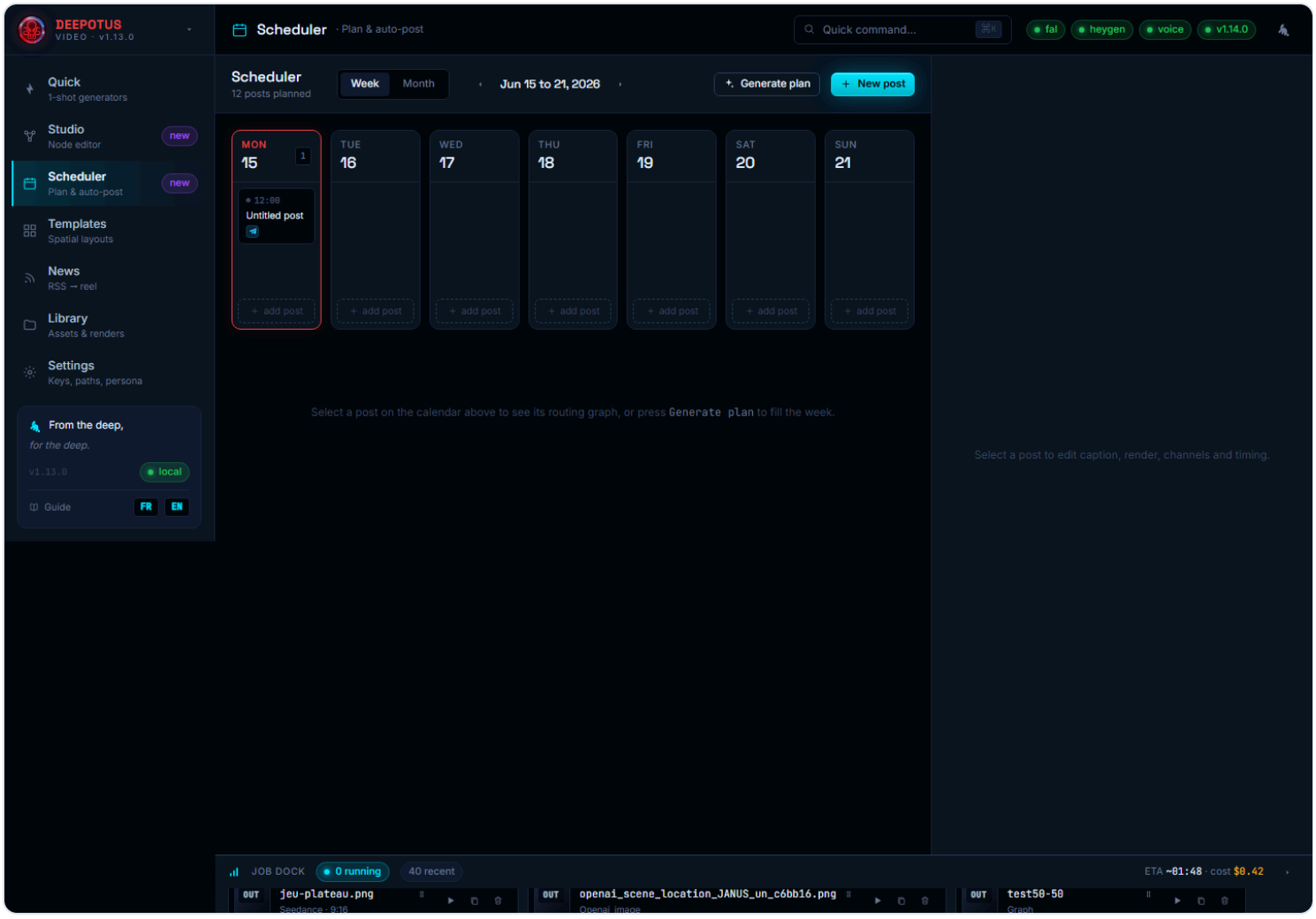
The basic gestures

1. **Start from a template:** in the palette, *Starter graphs* section, click "Seedance solo", "Avatar post", "UGC + animation", "News reel" or "Timeline montage".
2. **Start from scratch: New** button (top bar) → a blank canvas with just the *Render* output node. The right panel shows the **graph name**, editable.
3. **Add a node — drag & drop:** grab any entry from the palette (Image, Seedance, UGC video, Music track, Text…) and **drop it on the canvas**. The node appears where you dropped it.
4. **Wire:** drag from a node's output port (right side) to another's input port (left side). **Move:** drag the node body. **Delete:** the × in the node header (it also removes its connections).
5. **Check then run:** *Preview* shows inputs/the render; *Run* compiles the graph and launches the real render (follow it in the Job Dock).

> When you pick an asset in a node (a Library image, an existing render, your UGC clip, a music track), a **thumbnail fills the node**: you see at a glance what the chain will produce.

10 Schedule & publish your week EXPANDED

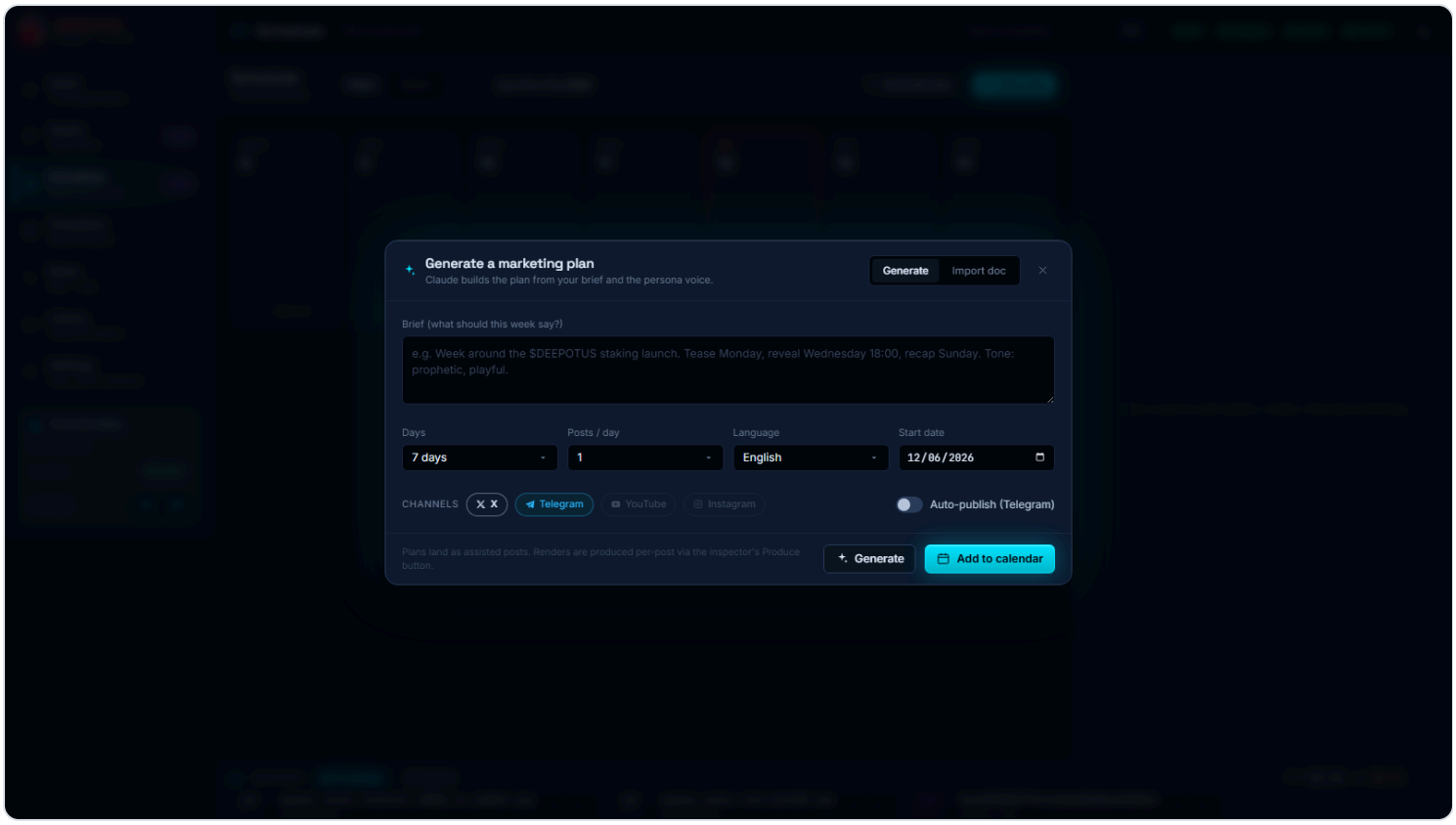
Scheduler tab: the publishing calendar, and the heart of "human-in-the-loop" — **nothing goes out without your approval** (unless you decide otherwise).



Scheduler: week/month, AI plan, document import, the post's graph + inspector on the right.

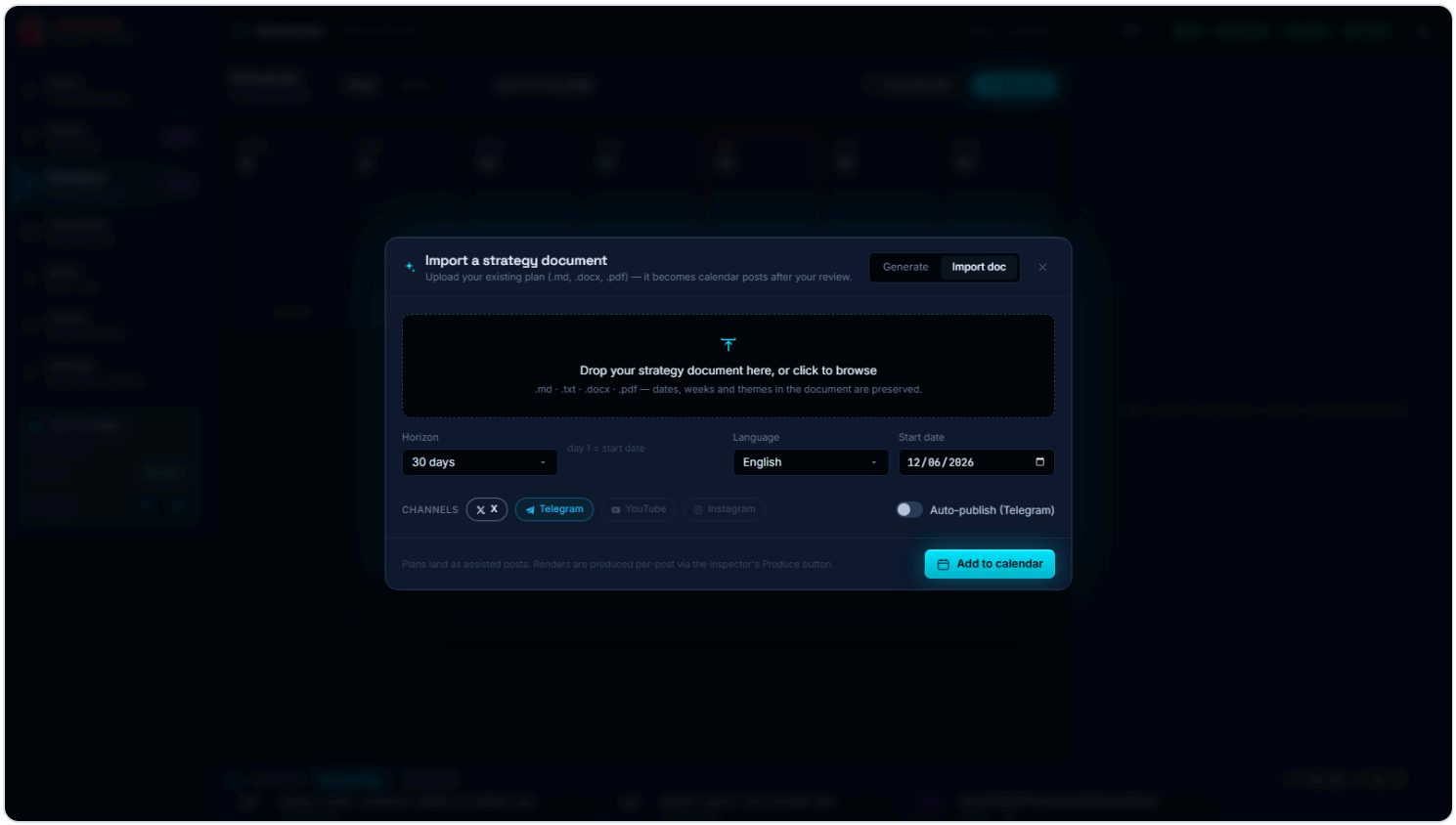
Three ways to fill the calendar

1. **Generate plan**: describe your week in one sentence → the AI proposes a full plan (dated posts, hooks, captions, formats). *Review the preview* before "Add to calendar".



"Generate plan": a one-sentence brief becomes a full plan, reviewed by you (human-in-the-loop).

2. **Import doc**: drop your existing plan (.md, .docx, .pdf) — transcribed into dated posts, **weeks preserved**.

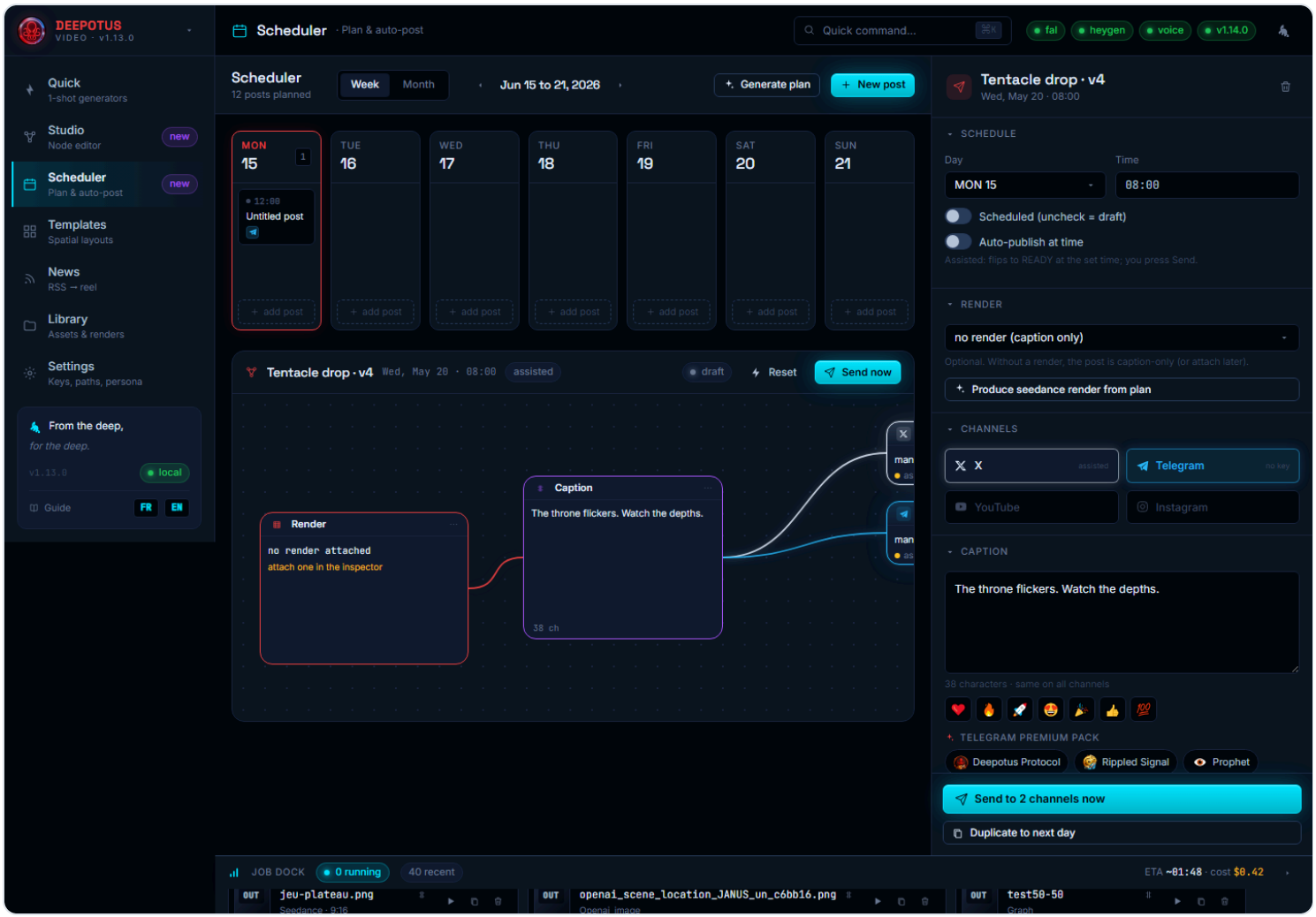


"Import doc": your strategy document becomes calendar posts.

3. **New post** or + *add post* on a day: manual creation. Episodes also land here as drafts (chapter 7).

Generate the render — before posting

Select a post: the right inspector opens the post's whole workshop. **Produce** button: the app builds the image (FLUX) then the clip (Seedance) from the plan's ideas — or reuses the source you chose. A **thumbnail** of the render appears on the post-graph's *Render* node, and a **video preview** shows in the inspector: you see the result **before** clicking Send or enabling Auto.



A post's inspector: channels, caption, and the **emoji bar** + the **Telegram Premium pack** (branded one-tap tags) — plus the render preview before publishing.

Polish the caption — emojis & Premium pack

Under the caption, two one-tap rows, ideal for Telegram and X:

1. **Quick emojis**: ❤️ 🔥 🚀 😍 🎉 👍 💯 — inserted at the cursor.
2. **Telegram Premium pack**: branded tags per content vertical (🐙 Deepotus Protocol, 🌊 Rippled Signal, 👁️ Prophet, 🎴 Board Game, 🎲 D&D, 🪙 $DEEP…) to sign each post in one gesture.

Auto or assisted?

Each post has a mode. **Assisted** (default): at the set time, the post flips to `READY` — you click *Send*. **Auto**: it goes out by itself on connected channels (Telegram, X). Status by color: gray = draft, cyan = scheduled, amber = ready, green = posted, red = failed.

---

11 Choose your AI engine (cloud & local) v1.15

Several features call a language model: **news summaries** (News), **marketing plan generation** (Scheduler) and **Episode storyboards** (the "By AI" split). As of v1.15, you choose the **provider** for each.

Cloud providers — Anthropic, OpenAI, Gemini

Paste whichever key(s) you want in **Settings** → **API keys**: **Anthropic** (Claude), **OpenAI** (GPT) and/or **Google Gemini**. Just **one** is enough to enable summaries and AI plans; add several if you want to switch between them.

> **Gemini model** v1.21: by default the app uses `gemini-flash-latest`, Google's stable alias that automatically tracks the latest Flash release — nothing to do. To pin a specific version (e.g. `gemini-2.0-flash`), fill the **"Gemini — modèle"** field in *Settings → API keys* and restart the backend. The same Gemini key also powers the Studio's native Google video models (Veo 3.1).

1. In **Settings** → **Provider defaults**, pick the provider per role (summarizer, planner) from a dropdown. A role is only marked "missing" when **none** of its possible providers has a key.
2. **Automatic fallback**: if the chosen provider is unavailable (no key, outage), the app moves to the next available one — you're never stuck.

> **Engine priority**: your choice in *Provider defaults* first, then the default order **Anthropic** → **OpenAI** → **Gemini** → **Ollama** → **built-in planner** (deterministic). The plan badge shows which engine actually ran.

Fully local LLM — Ollama (free, private)

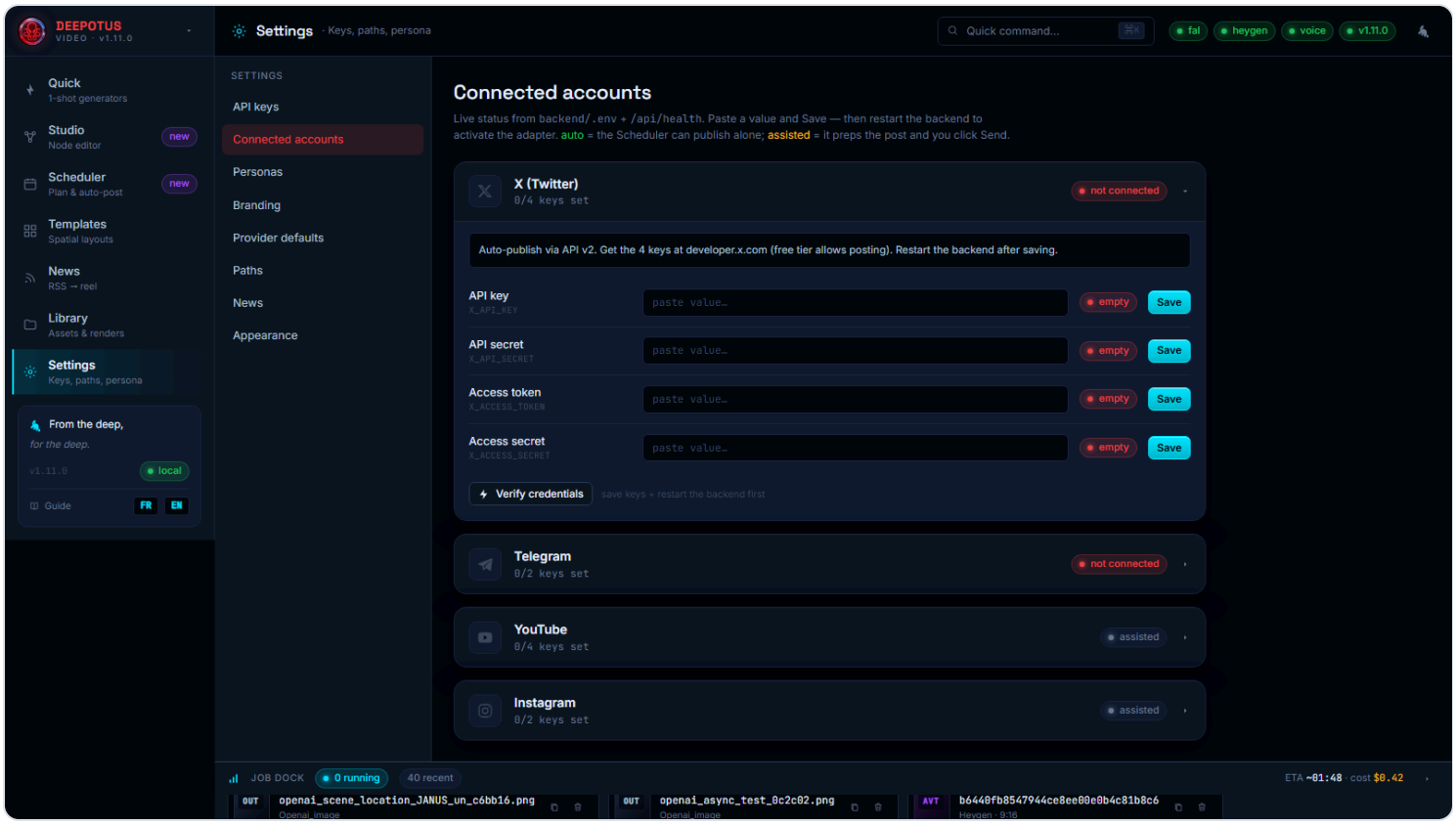
Plans can also run on an **AI model installed on your machine**: it's **free** and your strategy never leaves your computer.

1. Install **Ollama** from `ollama.com` (Windows).
2. Pull a model: `ollama pull qwen2.5:14b-instruct`. *Recommended: 14B or more.*
3. **Settings** → **API keys** → **Local LLM (Ollama)**: leave the default URL, set the *model name*, *Save*, restart the app.

---

12 Connect your accounts (publish)

Settings → **Connected accounts**: the credentials the Scheduler uses. All stored locally.



Connected accounts: real status per channel — auto (publishes alone) or assisted (you click Send).

Telegram (5 minutes, free, recommended)

1. On Telegram, talk to `@BotFather` → `/newbot` → copy the token.
2. Add the bot as an **administrator** of your channel.
3. Paste token + chat ID, *Save*, restart, then *Send test message*.

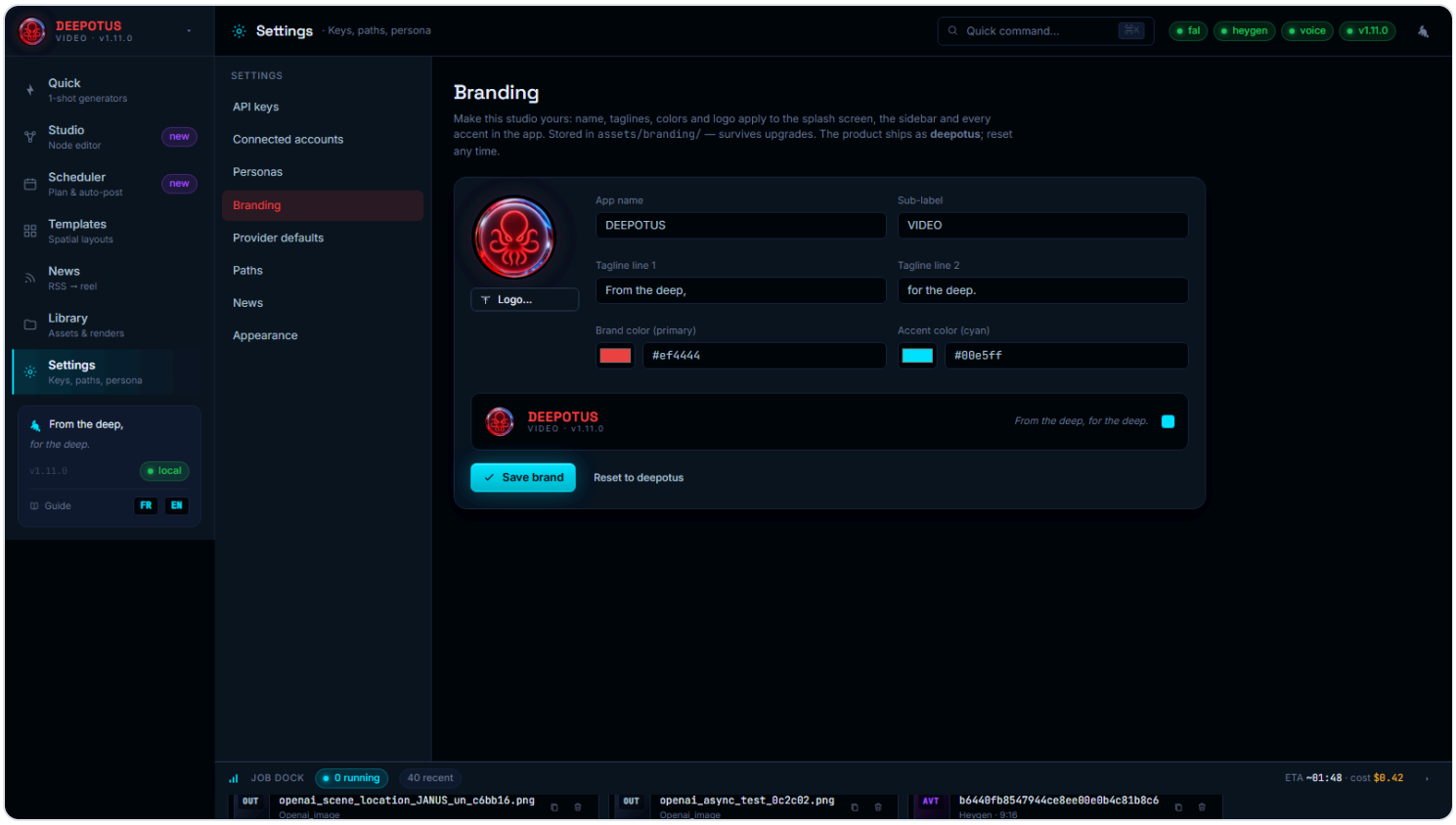
X (Twitter)

Create your 4 keys at `developer.x.com` (writes available pay-per-use) and paste them. *Verify credentials* button. **YouTube** and **Instagram** stay assisted.

---

13 White-label (rebrand the studio)

Settings → **Branding**: name, subtitle, taglines, colors and logo apply to the splash, the sidebar and every accent of the app.



Branding: live wordmark preview. Stored in the data folder — survives updates.

1. Edit the name, taglines, primary color and accent.
2. *Logo…* to upload your logo (auto-normalized).
3. *Save brand* — applied everywhere. *Reset to deepotus* restores the default brand.

> Your logo also appears on the **startup loading screen**: replace it here and your own logo greets users every launch — the studio becomes truly yours.

14 Troubleshooting

Symptom	Fix
`down` pill top-right	The engine isn't running: relaunch the Desktop icon.
Red pill on `fal` / `heygen` / `voice`	Missing/invalid key: Settings → API keys, fix, restart.
HeyGen avatars are slow to appear (or the list looks empty at first)	**Normal on the very first load**: HeyGen returns 1,000+ avatars and takes up to ~1 min to respond (their service is slow). The app pre-loads the list at startup and caches it — wait once, then it's instant. If the **HeyGen status** is green in Settings, your key is fine: it's neither the key, the network, nor the antivirus.
A voice gives a "paid plan" error	On the ElevenLabs free plan, only "premade" voices work via the API; custom library/community voices need a paid ElevenLabs plan. Pick a premade voice, or upgrade ElevenLabs.
After a restart, keys/library "gone"	No data is lost (it lives in `%LOCALAPPDATA%\DeepotusVideoGenData`). Relaunch cleanly via the Desktop icon.
The plan uses the built-in planner	No Anthropic key nor Ollama model (chapters 11–12).
`CERTIFICATE_VERIFY_FAILED` error	Antivirus inspecting HTTPS — handled automatically; otherwise disable HTTPS inspection.
"Port 8765 already in use"	The app is already running — open `http://127.0.0.1:8765`.
Where are my files?	Images: `…\DeepotusVideoGenData\assets\images` · Videos: `…\assets\outputs\final` · Your uploads: `…\assets\outputs\uploads` · Sounds: `…\assets\audio`.

🍳 Recipe A — a meme posted to Telegram in 5 minutes

1. **Library** → "Create image": describe the meme (~3 s).
2. **Scheduler** → *New post*: caption (+ a 🔥 emoji), Telegram channel, time in 5 min, *Auto* mode.
3. Or don't wait: **Send now**. Posted.

Recipe B — your selfie video + an animation (UGC)

1. Film 10–15 s on your phone, transfer the file to the PC.
2. **Studio** → "UGC + animation" starter → *UGC video* node → *Upload your video* (⏱ Duration master on).
3. Choose the image + prompt of the animated half (Seedance) → *Run*. The output lasts exactly your clip.
4. **Scheduler** → attach the render, polish the caption (emoji + pack), schedule.

Recipe C — today's news reel

1. **News** → *Refresh* → tick 2-3 articles.
2. *Build script* (persona voice) → *Build illustration*.
3. Compose in **Templates** → *Send to Scheduler* → schedule or publish.

Recipe D — an episode of your novel v1.15.6

1. **Episodes** → title + paste (or upload) the chapter → pick a voice & language → *Generate narration*.
2. **By AI** (or By paragraph) → review the scenes → *Generate all illustrations* → set each scene's motion (Ken Burns / Seedance).
3. *Assemble the episode* → preview → *Send to Scheduler* (YouTube + Instagram drafts). One episode a day, done.

---

15 Studio — Effects, masks & preview new

Two additions to the node editor (Studio).

The “Effects / Mask” node

Drag the **Effects / Mask** node (*Composition* section) and wire its output to the new **fx** port on the **Render** node. The panel shows a **thumbnail grid** so you pick an effect at a glance.

Available effects: LUT / Grade (named grades — Teal&Orange, Cyberpunk, Deep-sea, Noir, Vintage…), **VHS** (sequenceable line displacement, intensity + speed), **Colorize** (Sepia, Duotone, Matrix, Red-alert…), parametric **Gradients** (2 colors, angle,

opacity, blend) — plus modern classics: grain, vignette, chromatic aberration, glitch, bloom, halation, scanlines/CRT, cinematic letterbox, old film, sharpen, blur, dreamy, pixelate, camera shake, mirror, invert.

Under **"Apply to"**, choose *Whole render* (global post-pass) or **a specific node** (the targeted layer). Add several Effects nodes to stack effects or target different layers.

1. + Effect → click a thumbnail → set intensity (and preset / speed depending on the effect).
2. Wire the output to the Render **fx** port, choose the target, then *Preview* or *Run*.

"Preview" button — see it before you pay

The **Preview** button (top bar) produces a **free, fast** render of the final composition: not-yet-generated **Seedance/HeyGen** clips are replaced by their **source still**. You see the framing, layers, **overlays and effects** exactly as in the final render — *before* the paid **Run**.

> Text overlays (Text overlay / Ticker / Separator) now apply on **every** composition at the Render node (UGC, montage, spatial compose), not only spatial compose.

---

16 Moving to another PC — migration & export new

Everything is stored **locally**: the app in `%LOCALAPPDATA%\DeepotusVideoGen`, and your data (generations, **calendar + scheduled posts**, API keys) in `%LOCALAPPDATA%\DeepotusVideoGenData`. To continue on another PC (travel) **without interrupting the schedule**, a kit lives in `Desktop\deepotus-migration`.

With the scripts (recommended)

1. On THIS PC, with a USB/external drive plugged in: `export-migration.ps1 -Dest "E:\deepotus-transfer"`. Copies the app + all data + a consistent DB snapshot.
2. Copy that folder to the other laptop, then **double-click** `IMPORT-TOUT.cmd` at its root: it restores **everything in one click** — app, generations, calendar, keys, *and* your Claude Code sessions if they are in the kit. (Manual equivalent: `import-migration.ps1 -Src "E:\deepotus-transfer"`.) The `DeepotusVideoGen-Setup-*.exe` installer is also bundled for a clean install on a third machine.
3. Launch **Deepotus Video Gen**: *Library* = your renders, *Scheduler* = your scheduled posts, *Settings* = your keys.

Manually

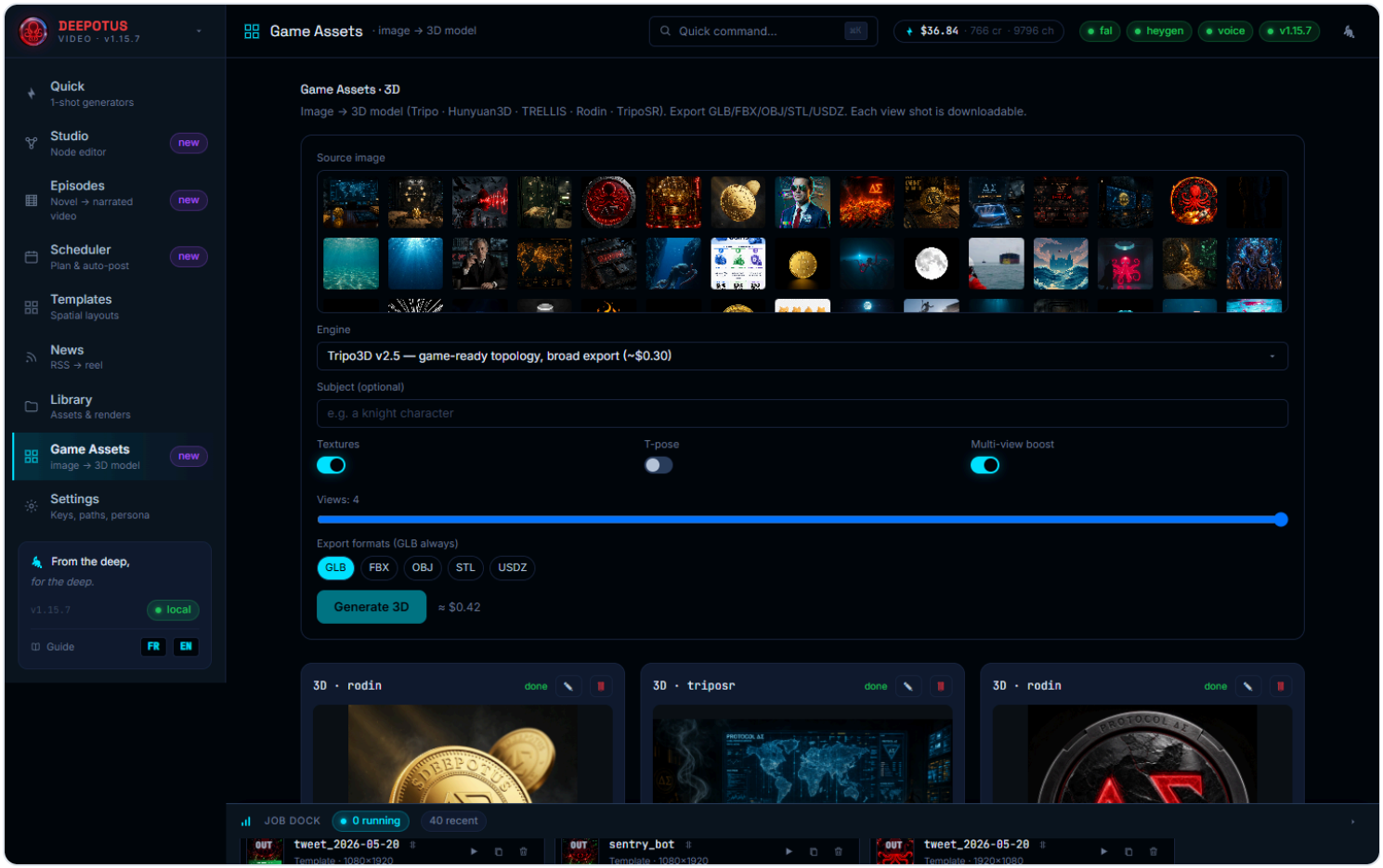
Close the app, copy the **two folders** above to the **same** `%LOCALAPPDATA%\` paths on the other PC, then run `scripts\create-desktop-shortcut.ps1` (or double-click `scripts\launch-silent.vbs`).

Do **not** run the app on both PCs at once (scheduled posts could double-publish). Best: **same Windows username** on both (paths match; otherwise the import script rewrites them). The **Scheduler runs while the app is open** — keep it running around posting times on the travel laptop, and keep the old PC closed.

---

17 Game Assets — image → 3D object new

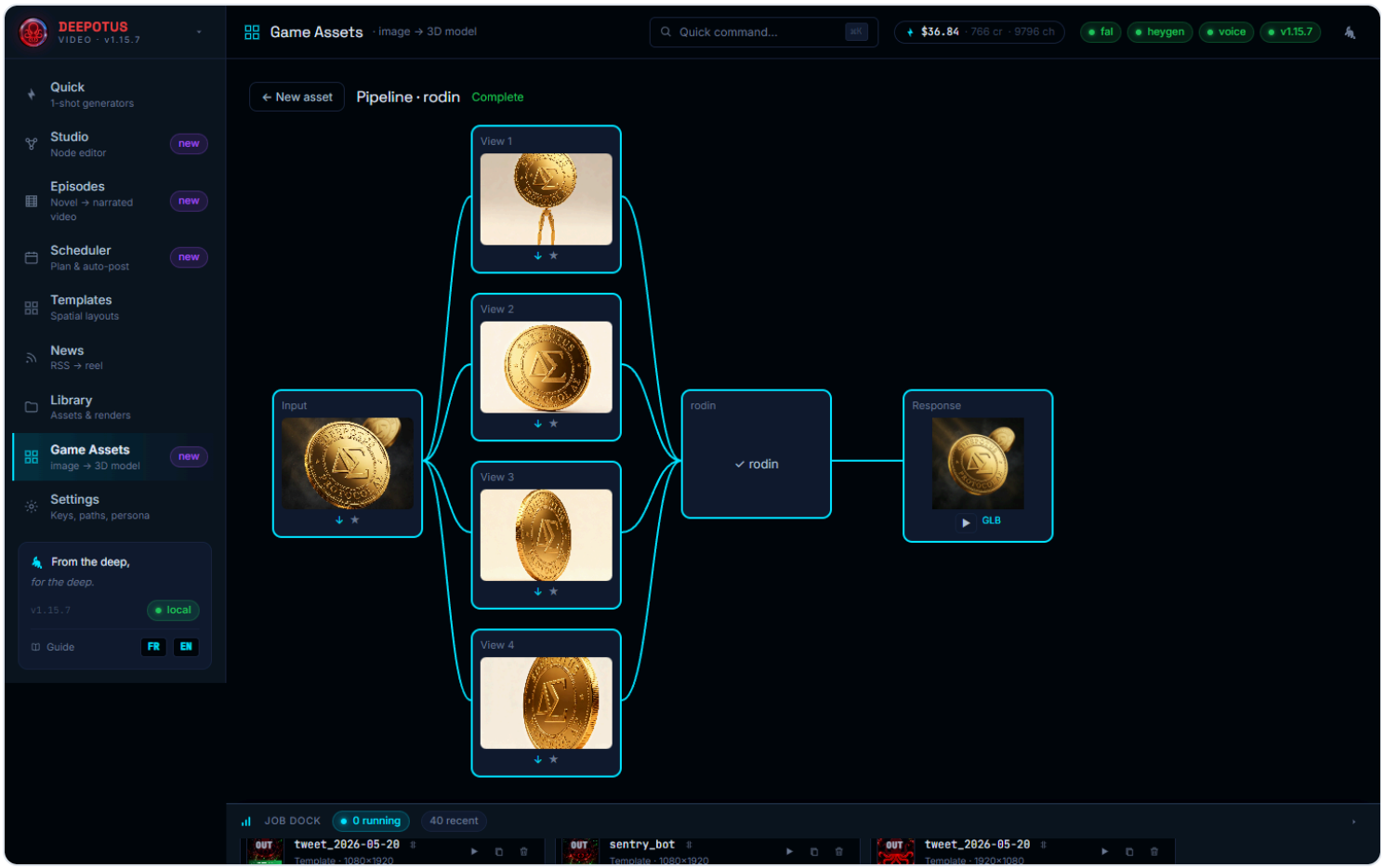
The **Game Assets** category (sidebar) turns an image from your Library into a **textured 3D model** ready for a game engine or 3D tool (Unity, Unreal, Godot, Blender…). Only the **fal.ai** key is required. Five generation engines, from cheapest to premium: **TripoSR** (≈ $0.07), **Hunyuan3D**, **Tripo** (default — best quality/price), **TRELLIS** and **Rodin**. The estimated cost is shown live before you start.



The Game Assets form: source image, engine (with description and rate), subject, options and export formats.

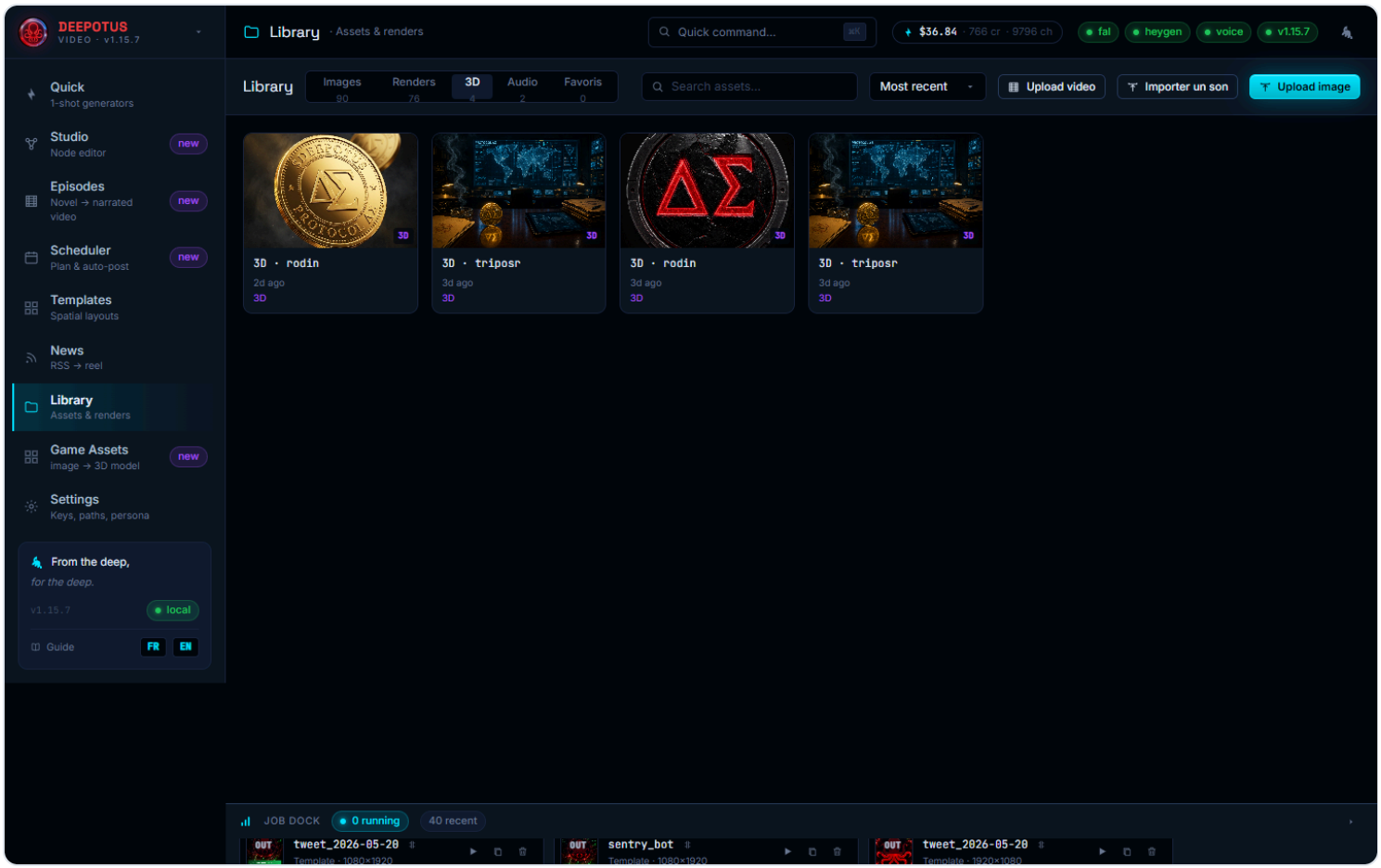
1. Pick a **source image** (your Library images — ideally a well-isolated character/object on a neutral background), then the **engine** from the dropdown.
2. Options: **Multi-view** (on by default, 4 views) first generates other angles via Seedream for a more faithful mesh; **Textures**; **T-pose** for characters you plan to rig; and the **export formats** — GLB always included, plus FBX / OBJ / STL / USDZ depending on the engine. A format the chosen engine cannot emit is flagged; each extra format can cost one more generation (the estimate shows it).

3. **Generate** opens the **pipeline canvas**: the Input → Views → Engine → Response nodes animate and fill in live (also visible when reopening an asset via its card title).



The live pipeline: each node fills in as the generation progresses (views, engine, result).

4. On the result card: ▶ **Rotate** spins the 3D model in the built-in viewer; each **format** downloads in one click; the **Shots** strip lets you download each view or send it to *Library* → *Images* (★) to reuse as a source. Rename (✎) or delete (🗑) the asset from its card header.
5. All your assets also live in **Library** → **3D tab**: preview, full-size 3D rotation, GLB download, favorites ★, rename and delete.



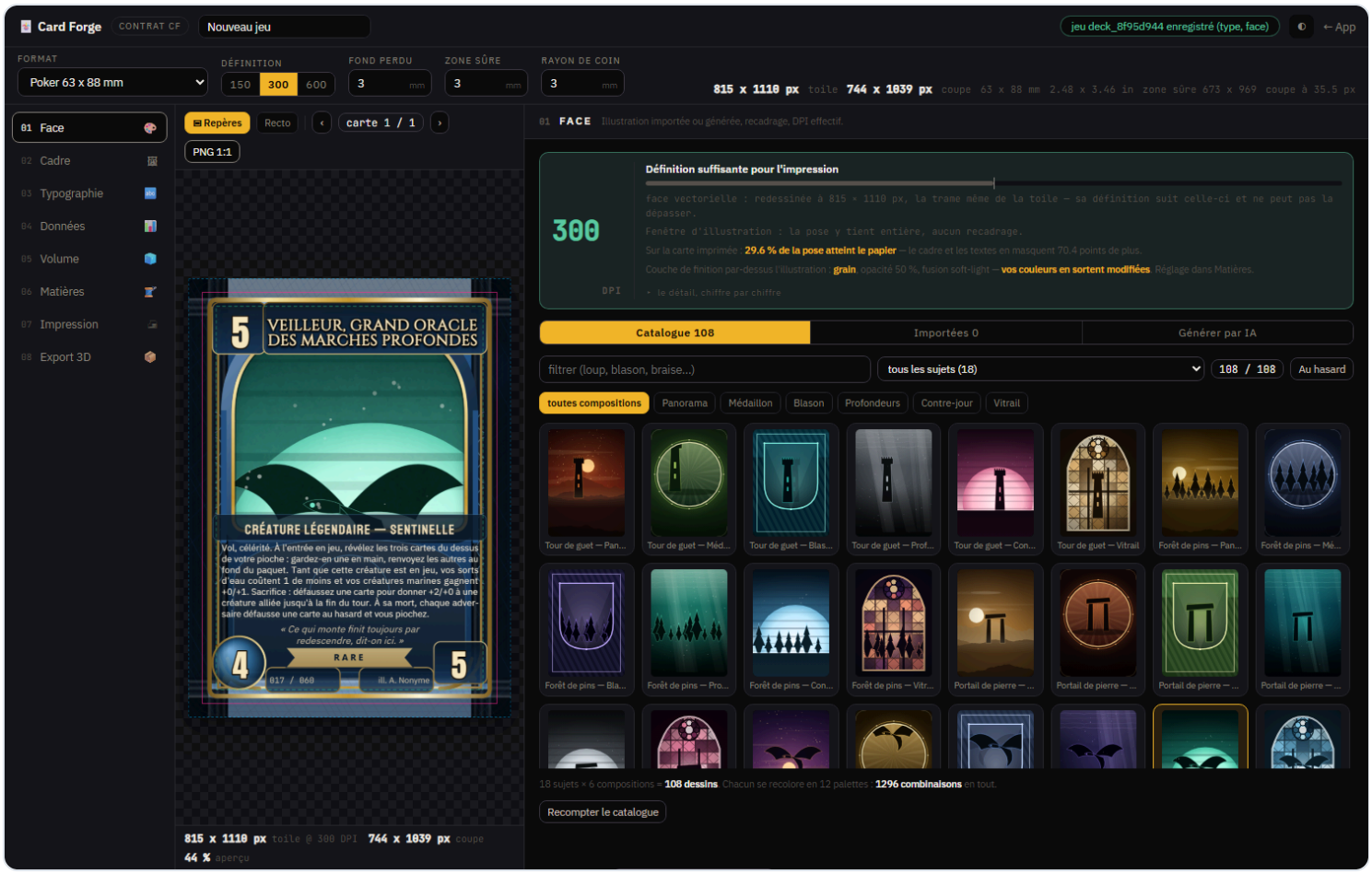
The Library 3D tab gathers all your generated assets.

3D rendering runs in your browser (WebGL): keep the tab in the foreground for smooth rotation. Files live under `DeepotusVideoGenData\assets\outputs\assets3d\` and follow your migration (chapter 16).

---

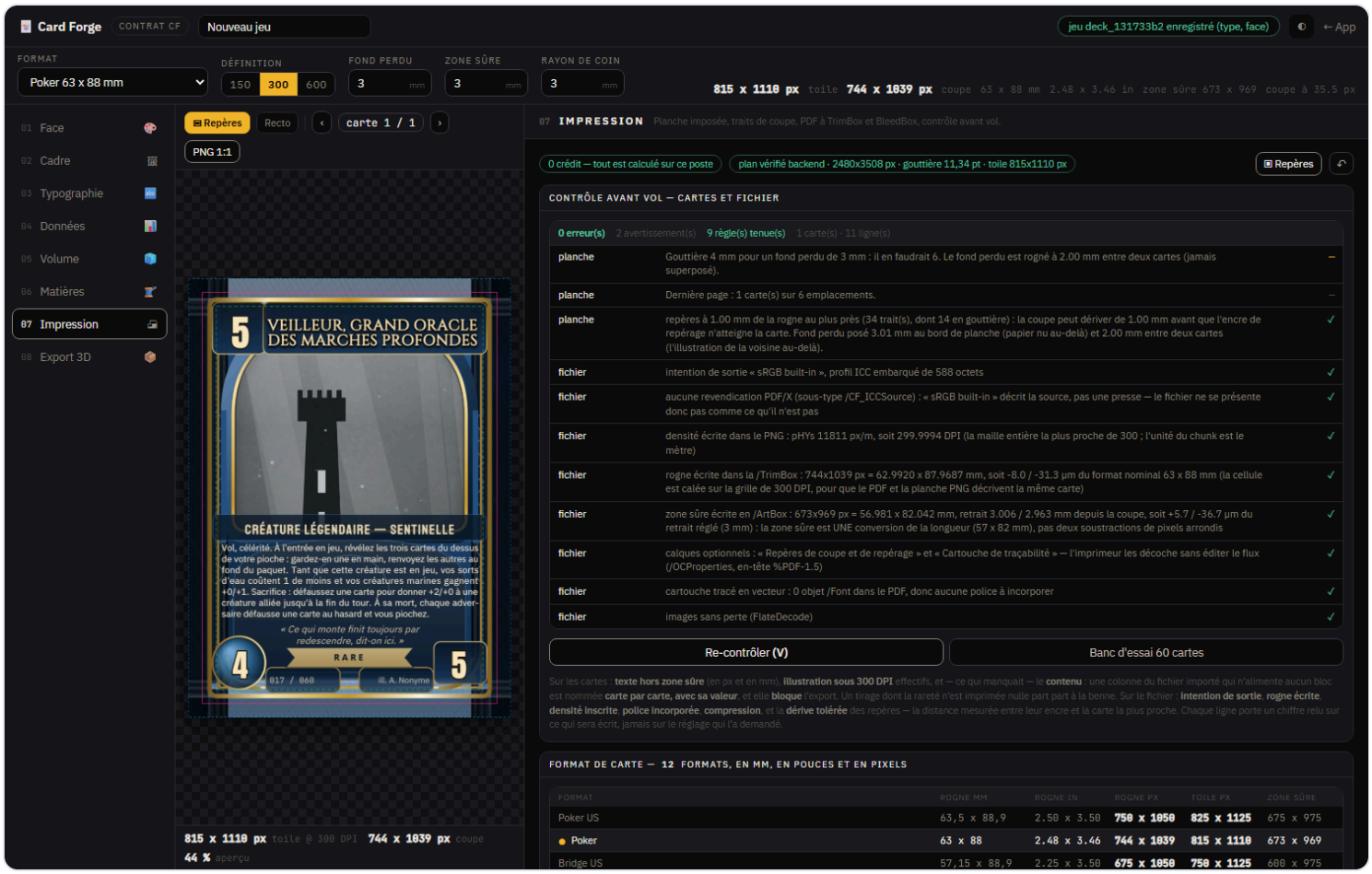
18 Card Forge — playing-card editor

The **Card Forge** sub-tab (inside **Game Assets**, next to 3D / Sprites / Tiles / Materials) is a full playing-card editor: face, frame, typography, CSV data, 3D volume, PBR materials, print export and 3D export — eight modules in one document. No key is required; only **AI face generation** uses your existing fal.ai key.



The Face module: a local catalog of 108 illustrations, import, AI generation, and a live effective-resolution gauge.

1. At the top of the lab, pick the **format** (poker, bridge, tarot, domino…), the **resolution** (150 / 300 / 600 DPI), the **bleed** and the **safe zone** — these settings apply to all eight modules and to the final export.
2. Browse the modules in the left rail: **01 Face** (imported illustration, AI-generated, or picked from the local catalog), **02 Frame** (borders, rarity, card back), **03 Typography** (title, rules box, flavor text, 23 already-bundled fonts), **04 Data** (CSV import → columns to fields → full deck), **05 Volume** (3D preview with real thickness and edge), **06 Materials** (8 PBR texture channels computed locally).
3. **07 Print** runs a **preflight check** on both the sheet and the produced file: bleed, cut marks, the PDF's `TrimBox` / `BleedBox`, the PNG's `pHYs` density (300 DPI = 11811 px/m) — every line is verified against the delivered file's bytes, never against an on-screen setting.



The preflight check: 0 errors, TrimBox and BleedBox present in the PDF, PNG density verified byte by byte.

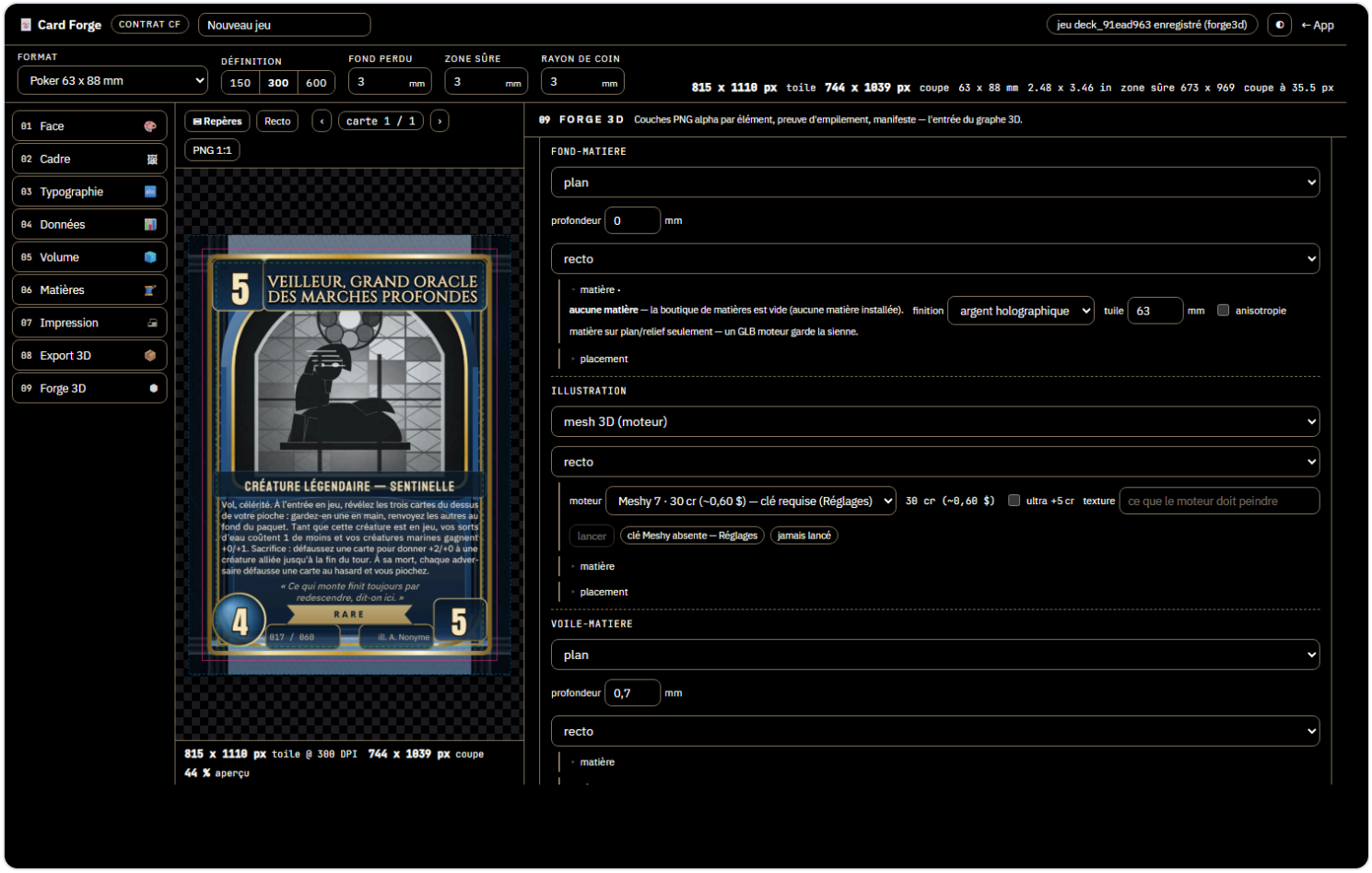
4. **08 3D Export** exports both a `.glb` **and** a standalone `.gltf` with the full PBR map set (base color, normal, roughness, metallic, occlusion, height, emissive, ORM), ready for Unity, Unreal, Godot or Blender.

> The **emissive** channel is now adjustable in the 3D export (an **Emission 0..1** slider — luminescent ink on matte paper, or extinguished gilding) and wired into the GLB, glTF and MTL; it stays capped at 1.0 and its mask is derived from the card's luminance (adjustable threshold). A pose hidden by the frame is fixed **in one click** from the Face module — a panel button plus a contextual offer next to the masking percentage.

---

19 Forge 3D — layers, graph & image→3D engines new

Card Forge's **09 Forge 3D** module turns your card into a printable 3D piece or an "NFT-grade" artifact: the card exports **as layers** (base material, illustration, veil, frame, typography, ornaments + the composite), each layer is processed in a **graph**, and assembly produces a clean GLB — plus an STL whenever everything is watertight. The whole free circuit runs locally, no key needed.



The Forge 3D graph: a material chain (silver holographic finish) on the background, an image→3D engine on the illustration — 30 cr (~$0.60) shown before any launch.

1. **Export the layers**: the screen proves the stacking **before** uploading (0 px of difference between the composite and the layer pile), the server counter-proves it in PIL and seals a manifest. An empty layer is still written — absence is measured, never guessed.
2. **Edit the graph**: each layer gets a **plane** treatment (flat card) or a **relief** (a grid displaced by luminance, a solid closed by construction), a **material** from the Material Forge shop (tiled at physical scale, in millimeters), a **silver or gilded holographic** finish (iridescence, clearcoat, anisotropy — verified in the GLB's bytes) and a **transform** (position, rotation, scale, z stacking).
3. **Image→3D engines** (optional, paid): the **mesh 3D** treatment sends the layer to one of 7 engines — tripo, hunyuan, trellis, rodin, triposr (fal, priced in $) or **Meshy 6/7 through the direct API** (credits; ultra +5 cr on meshy-7). The price shows on the node **and** in the graph footer before any launch; the status chip follows the job (queued → running → served · real credits → or failure with the literal message); relaunching a node resets its folder.
4. **Build**: assembly merges the engines' GLBs at their layer's place (full reindexing, the engine file's identity is dropped), writes the final GLB and the **STL** — accepted only when every element is a closed solid, with a motivated refusal otherwise. The docket lists the elements, their engines and the credits consumed.

The image→3D engines are the only paid part of Card Forge: fal bills in $ on your existing key, Meshy in credits on yours (`MESHY_API_KEY`, the Settings field shared with the 3D Studio). To try everything without spending a credit, set `MESHY_MOCK=1` in the data `.env`: the local simulator runs the full circuit, fake credits included.

---

20 Print your creations — 3D printing new

Everything the app produces in 3D — and even your vector illustrations — exports to your printer in one gesture. The circuit is designed for the **Elegoo Centauri Carbon 2** (256 × 256 × 256 mm build plate) and its slicer **ElegooSlicer** (bundled with the machine, an OrcaSlicer fork — OrcaSlicer works too). Every export writes a folder under `DeepotusVideoGenData\assets\print3d\` with three files: the **STL** (the mesh), the **3MF** (the one you open — it carries the millimeter unit and the name), and `impression.json` (provenance, scale, watertightness). Everything is local and free: no provider calls.

1. **Install ElegooSlicer once.** The installer associates `.3mf` files with the slicer on Windows: the app's "Open in slicer" button uses that directly. Without an association the app tells you so, and you can set `SLICER_PATH` (the slicer executable path) in the data `.env`.
2. **From the card Forge 3D** (Card Forge, module 09): when the build slip delivers an STL — that is, when every element is a **closed solid**, the check that guarantees it — a → **3D printing** button appears next to it. Leave the size empty to keep the card's real millimeters, or type a target (e.g. `80` for a figurine). These exports are marked *guaranteed* watertight: the producer proved it.
3. **From Game Assets · 3D**: every generated model (card characters, props) carries its → **3D printing** button. Give the largest dimension in millimeters — 80 for a figurine, 250 for a board — and the app centers the piece and rests it on the bed. The converter reads the original `model.glb`; the "optimized" file (compressed for game engines) is refused with a message that says what to use instead.
4. **From the Vectorlab** (Export menu → 3D printing…): your vector illustrations become **printable reliefs**. Give a global height in millimeters, then override per layer with `name=mm` — for example `2, contours=5` for a stained-glass piece: glass at 2 mm, leads at 5. Strokes without a fill count by their stroke width; text objects are skipped (and counted in the message). Flat dimensions follow the document's unit (an A4 comes out at 210 × 297 mm).
5. **Slice in the slicer.** The `.3mf` opens at the right size, in millimeters, resting on the bed. Supports, fine orientation, infill, filament profiles: that's the slicer's job — the app never generates G-code. Beyond 256 mm the export warns you the slicer will have to cut or shrink, without blocking you.

Meshes coming from the image→3D engines are marked *unknown* watertightness in `impression.json`: ElegooSlicer and OrcaSlicer repair small defects on import and say so. Only the card Forge 3D guarantees its solids — its "closed solid" check states it, not a promise.

Deepotus Video Gen v2.6.0 — guide generated from the real application. Everything runs locally on your machine; only provider calls (fal.ai, HeyGen, ElevenLabs, Anthropic) and your publications leave your computer. With Ollama, even plans stay local. · Version française